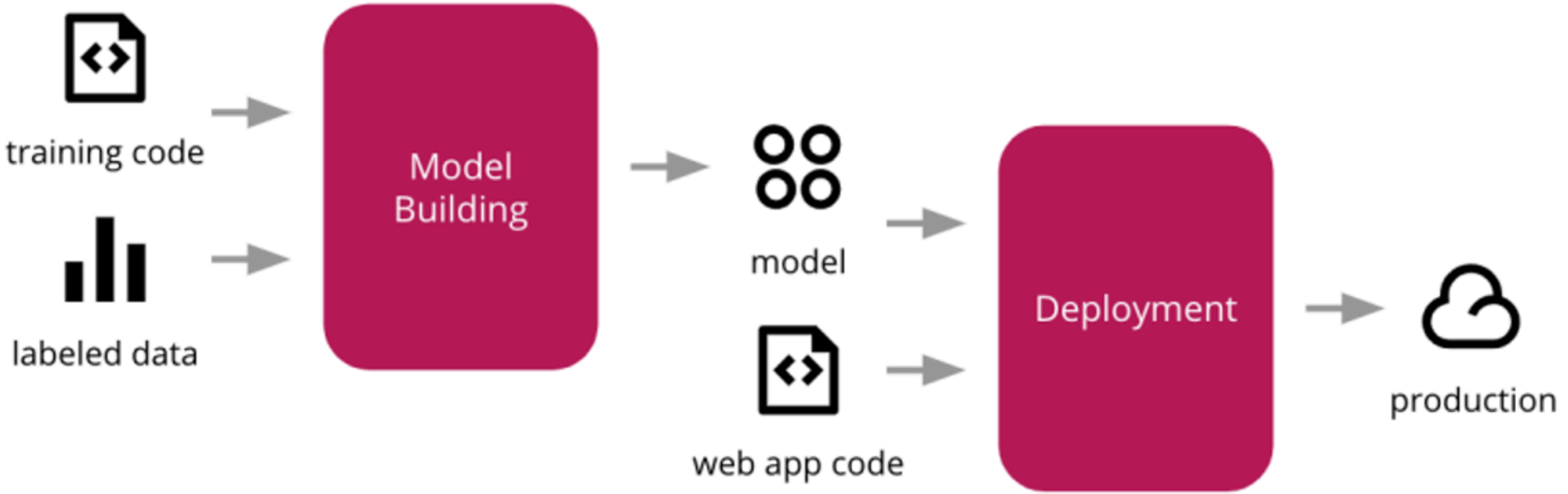
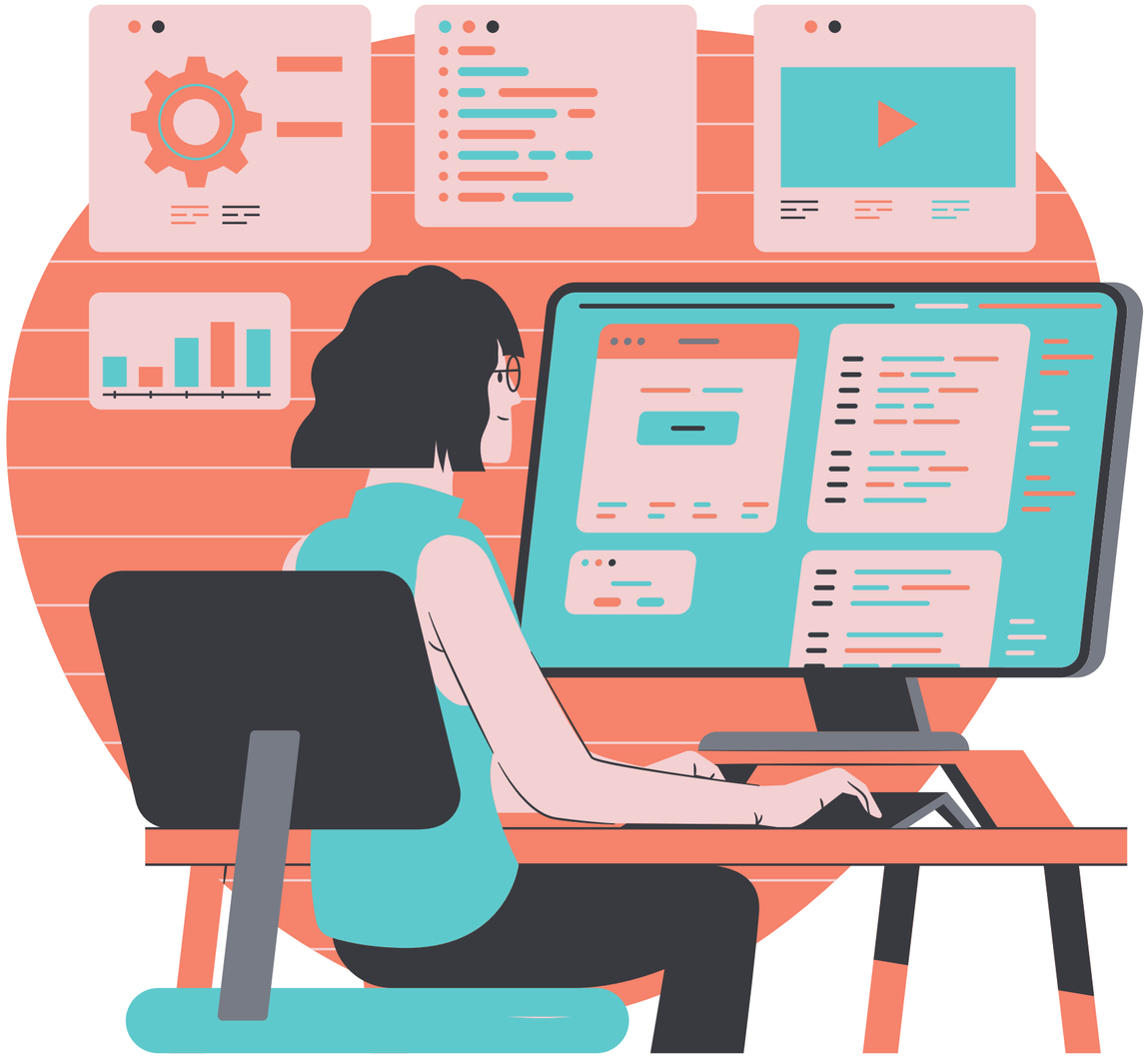
퍼셉트론와 선형한계



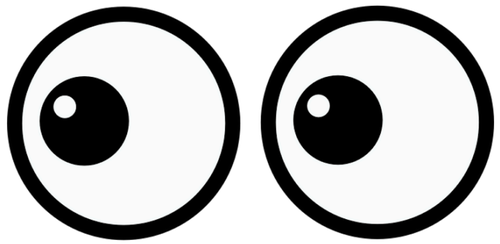
데이터를 가지고 모델을 학습하고, 자동으로 의미있는 정보를 얻는다.



감각

판단

행동



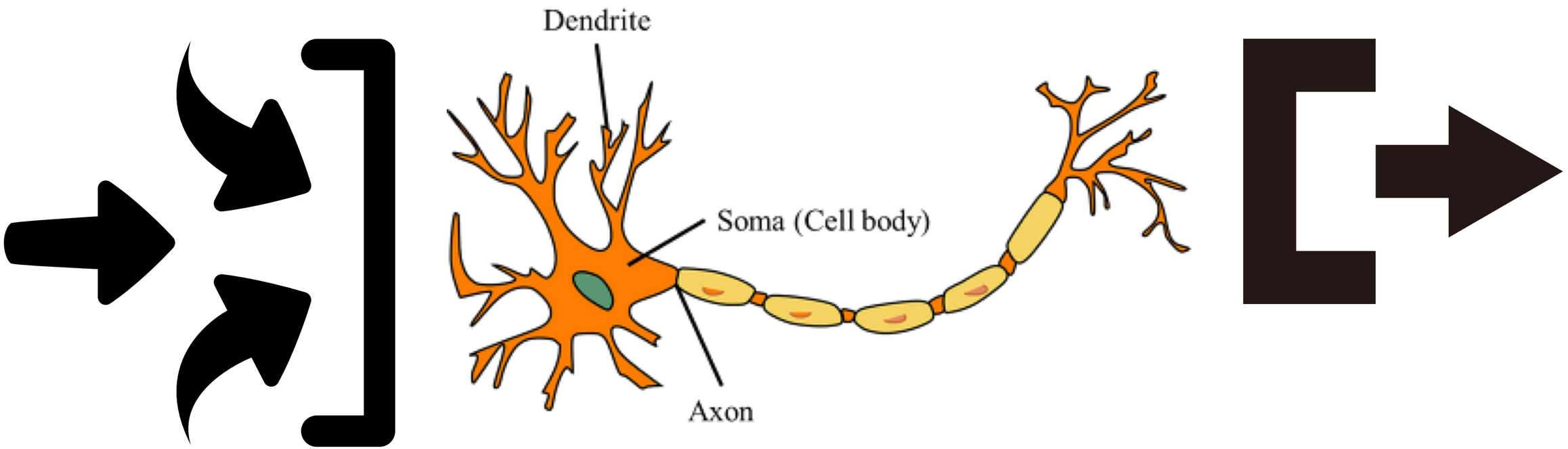


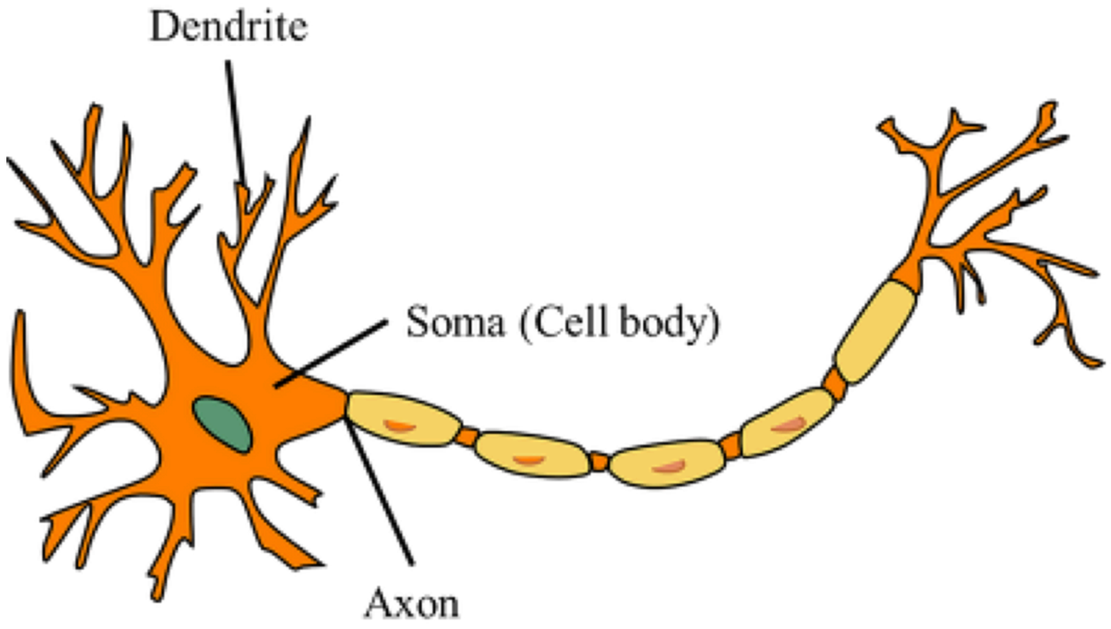


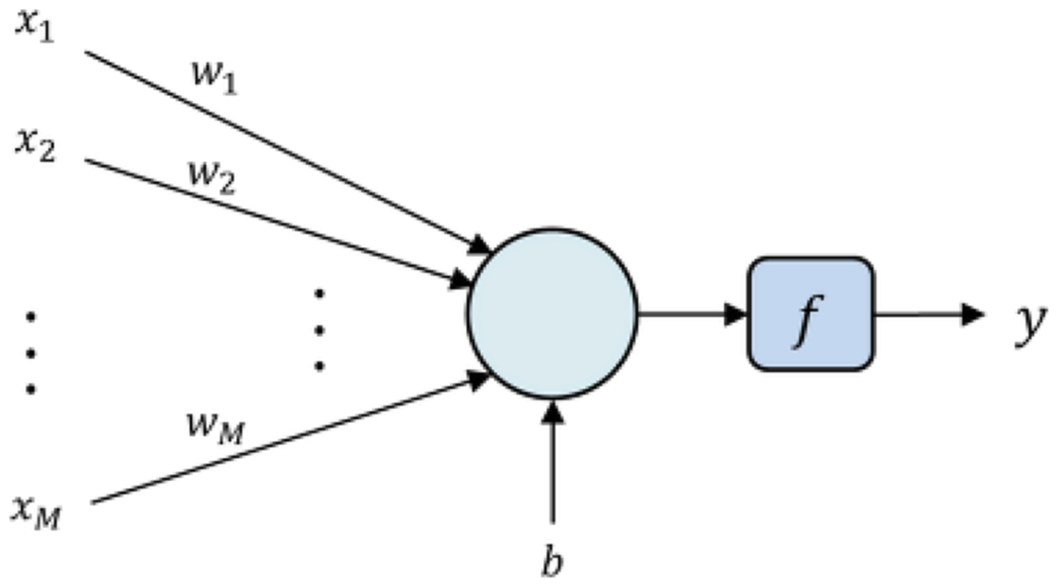
입력

처리

출력


Dendrite
Soma (Cell body)
Axon

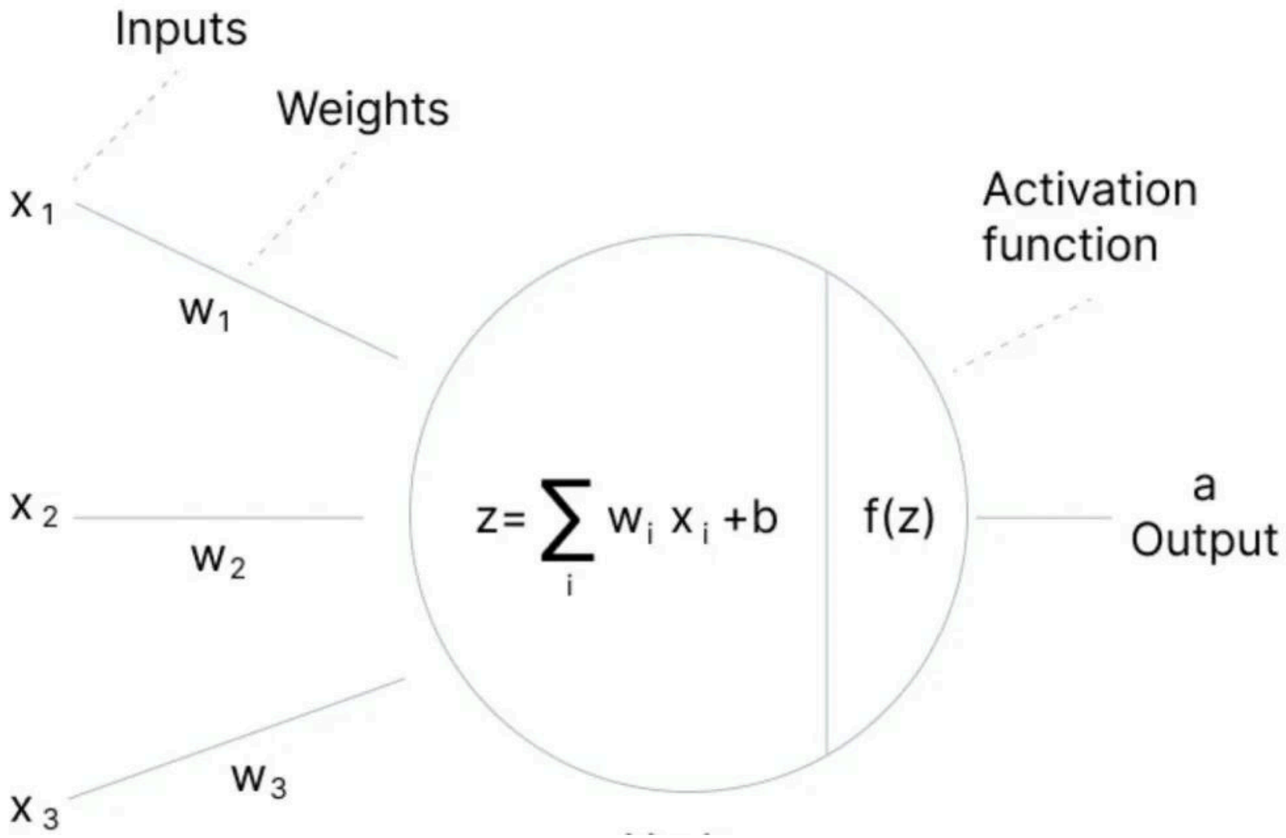


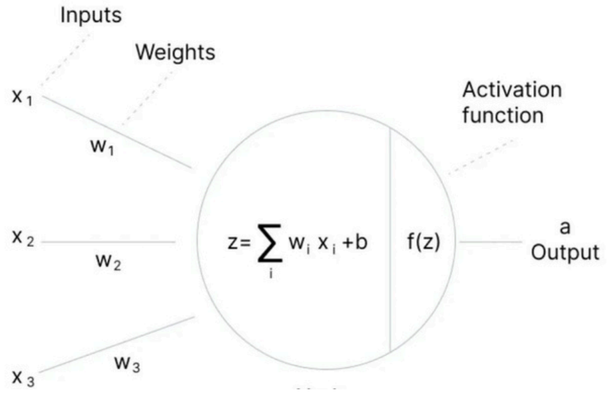


$$y = \begin{cases} 0\ (w_1 x_1 + w_2 x_2 \leq \theta) \\ 1\ (w_1 x_1 + w_2 x_2 > \theta) \end{cases}$$

Perceptron

Briefly elaborate on what you want to discuss.





선형 결합

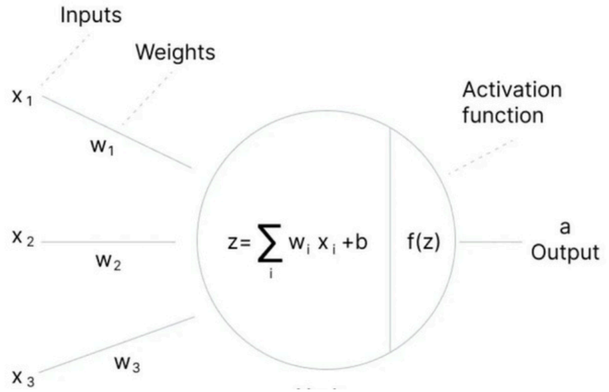
벡터공간 V에 속한 부분집합 $S = \{v_1, v_2, \dots, v_n\}$ 의 원소인 벡터 $v_1, v_2, \dots, v_n$와 어떤 스칼라 $a_1, a_2, \dots, a_n$에 대하여 다음을 만족시키는 벡터 $v \in V$를 S의 **선형 결합(linear combination)** 이라 한다.

$$v = a_1 v_1 + a_2 v_2 + \cdots + a_n v_n$$

선형 모델

변수(samples)들을 파라미터들을 계수로 선형결합한 구조

$$y = b + w_1 x_1 + \cdots + w_k x_k \qquad \mathbf{w} = \begin{bmatrix} b \\ w_1 \\ \dots \\ w_k \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ \dots \\ x_k \end{bmatrix}$$



선형 변환 (Linear Transformation)

일정한 조건을 만족하면서, 벡터의 공간(차원)을 변환시키는 행위

n차원 벡터를 m차원 벡터로 (선형)변환시키려면?

$$\begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} \longrightarrow \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \dots \\ y_m \end{bmatrix}$$

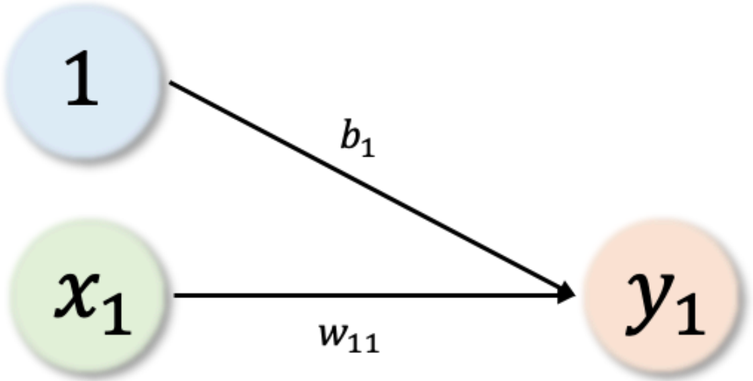
n차원 → m차원

m x n 행렬을 n차원 벡터 앞에 곱한다!

$$\begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \dots \\ y_m \end{bmatrix}$$

m x n 행렬 · n차원 · m차원

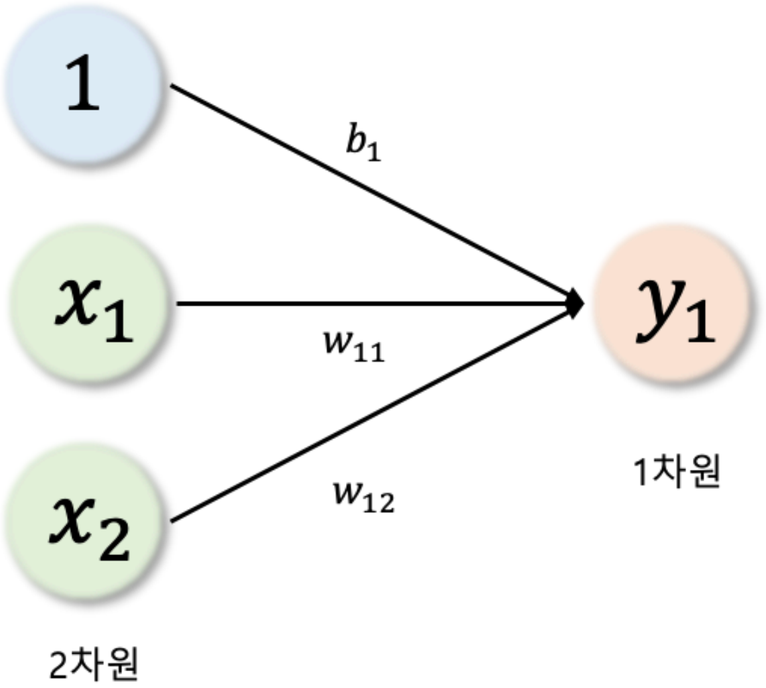
$$y_1 = w_{11}x_1 + b_1$$



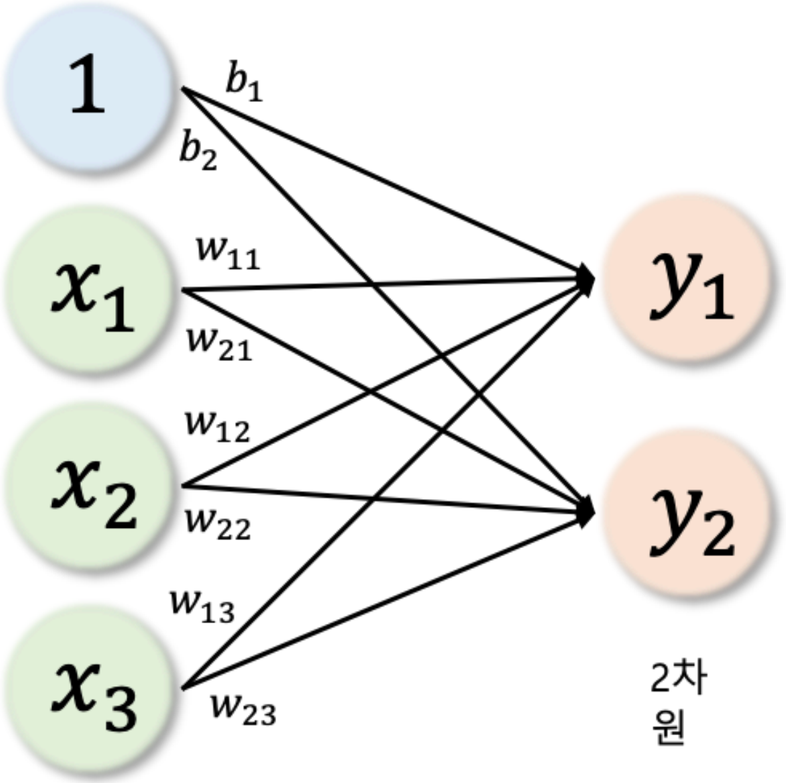
1차원 1차원

$$[w_{11}][x_1] + [b_1] = [y_1]$$

$$y_1 = w_{11}x_1 + w_{12}x_2 + b_1$$



$$[w_{11} \quad w_{12}] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + [b_1] = [y_1]$$

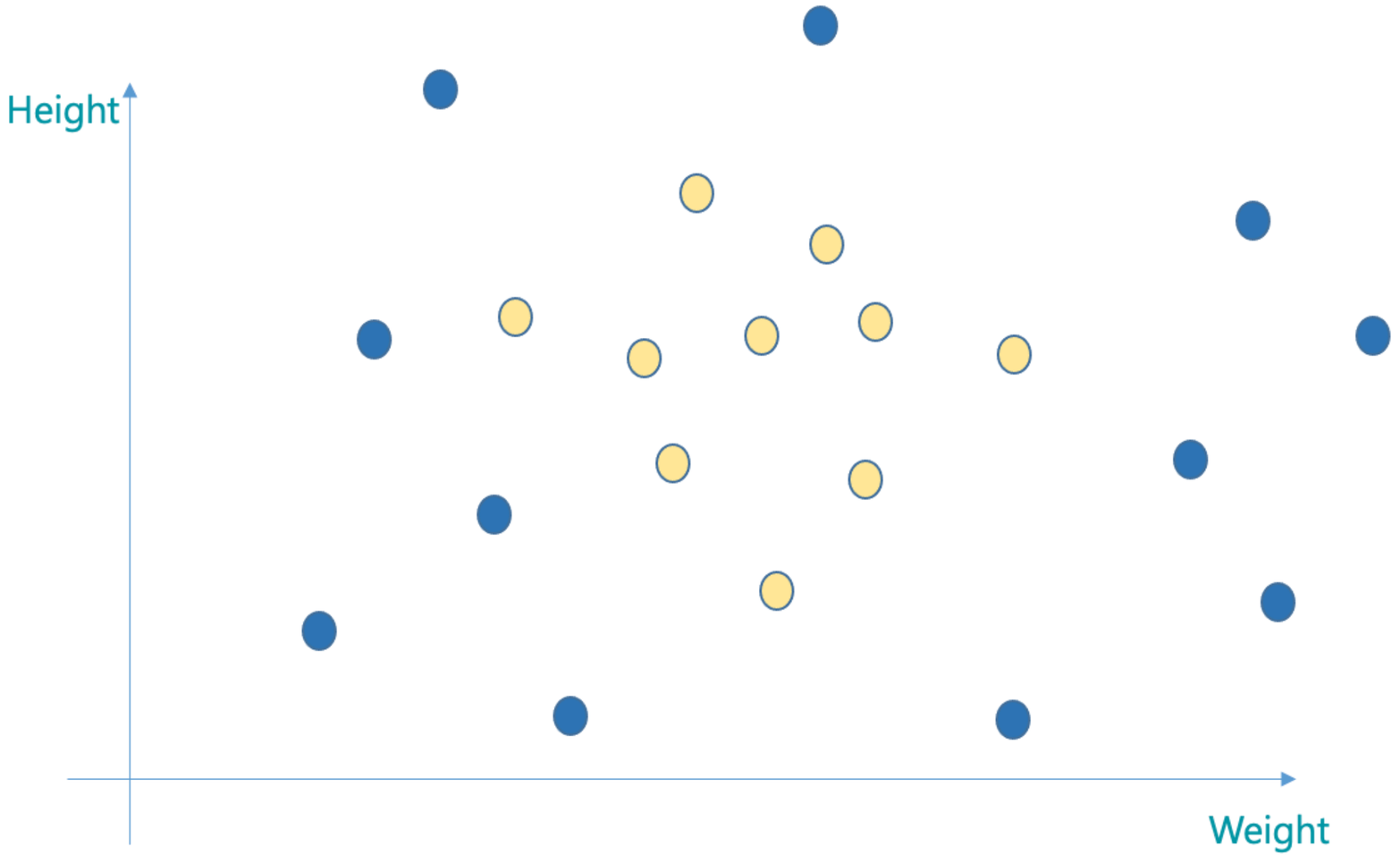


$$y_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1$$
$$y_2 = w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2$$

$$\begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

그래봐야

Y=AX+B



만약 데이터가 이렇게 분포되어 있다면

하나의 직선으로 분류할 수 있을까?

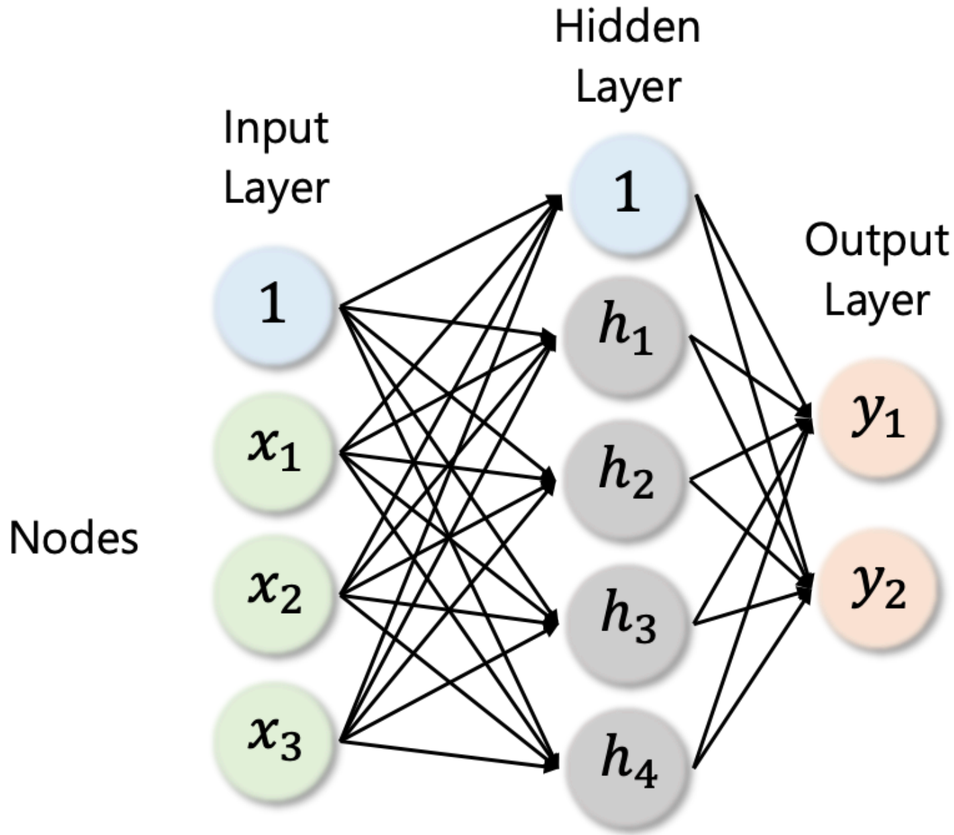
그래봐야

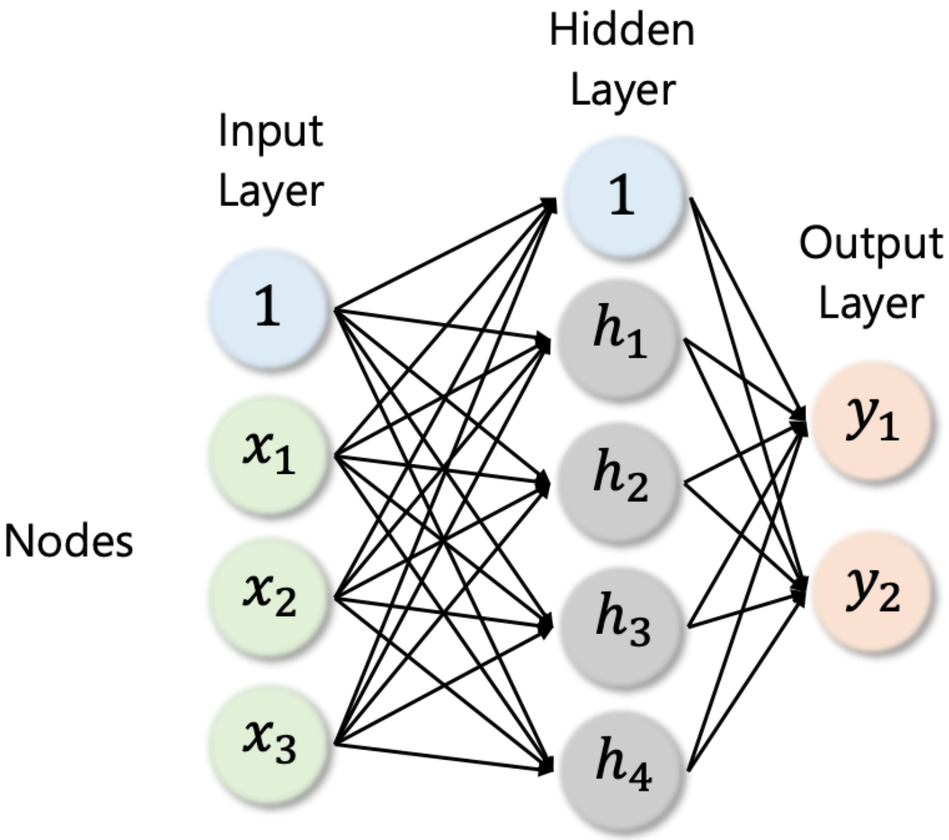
Y=AX+B

AI 겨울! XOR 문제!

활성화 함수

좀 더 복잡하게 만들면?


Hidden
Layer
Input
Layer
Output
Layer
Nodes
1
x_1
x_2
x_3
1
h_1
h_2
h_3
h_4
y_1
y_2



$$h_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1$$
$$h_2 = w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2$$
$$h_3 = w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + b_3$$
$$h_4 = w_{41}x_1 + w_{42}x_2 + w_{43}x_3 + b_4$$

+

$$y_1 = w'_{11}h_1 + w'_{12}h_2 + w'_{13}h_3 + w'_{14}h_4 + b'_1$$
$$y_2 = w'_{21}h_1 + w'_{22}h_2 + w'_{23}h_3 + w'_{24}h_4 + b'_2$$

=

$$y_1 = w'_{11}(w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1) + w'_{12}(w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2) + w'_{13}(w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + b_3) + w'_{14}(w_{41}x_1 + w_{42}x_2 + w_{43}x_3 + b_4) + b'_1$$

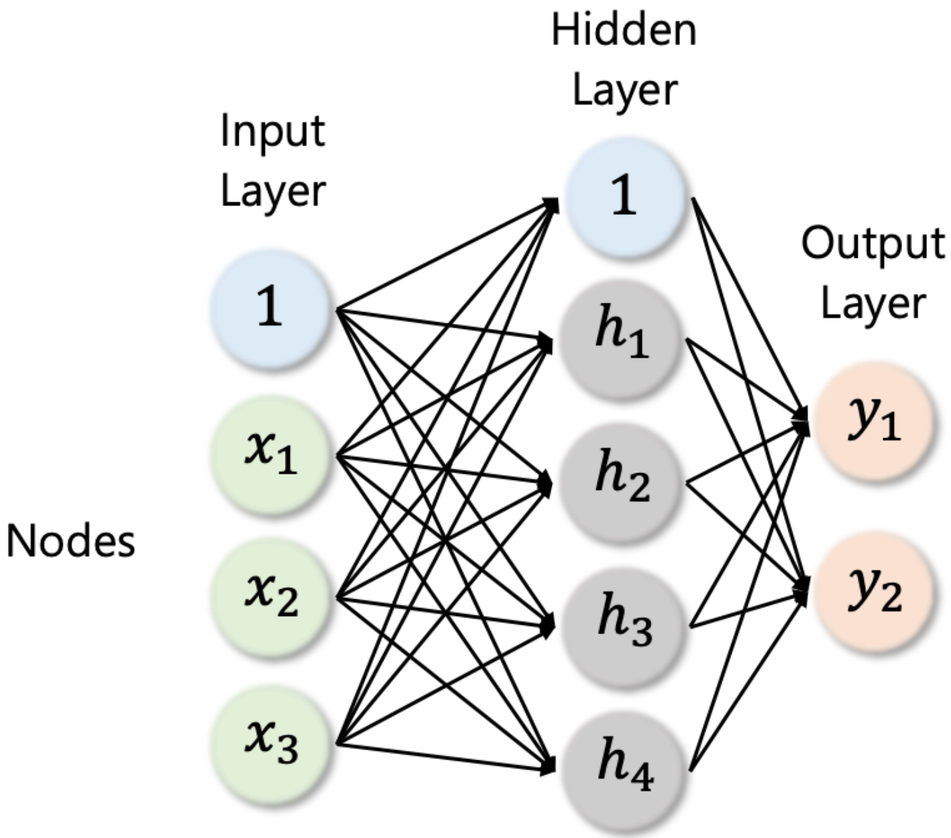
$$y_2 = w'_{21}(w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1) + w'_{22}(w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2) + w'_{23}(w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + b_3) + w'_{24}(w_{41}x_1 + w_{42}x_2 + w_{43}x_3 + b_4) + b'_2$$

결국 $y = \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_4 + \alpha$ 꼴

(아무리 쌓아도 선형)

그래봐야 Y=AX+B

이렇게나 복잡한데?



$$h_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1$$
$$h_2 = w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2$$
$$h_3 = w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + b_3$$
$$h_4 = w_{41}x_1 + w_{42}x_2 + w_{43}x_3 + b_4$$

+

$$y_1 = w'_{11}h_1 + w'_{12}h_2 + w'_{13}h_3 + w'_{14}h_4 + b'_1$$
$$y_2 = w'_{21}h_1 + w'_{22}h_2 + w'_{23}h_3 + w'_{24}h_4 + b'_2$$

||

$$y_1 = w'_{11}(w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1) + w'_{12}(w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2) + w'_{13}(w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + b_3) + w'_{14}(w_{41}x_1 + w_{42}x_2 + w_{43}x_3 + b_4) + b'_1$$

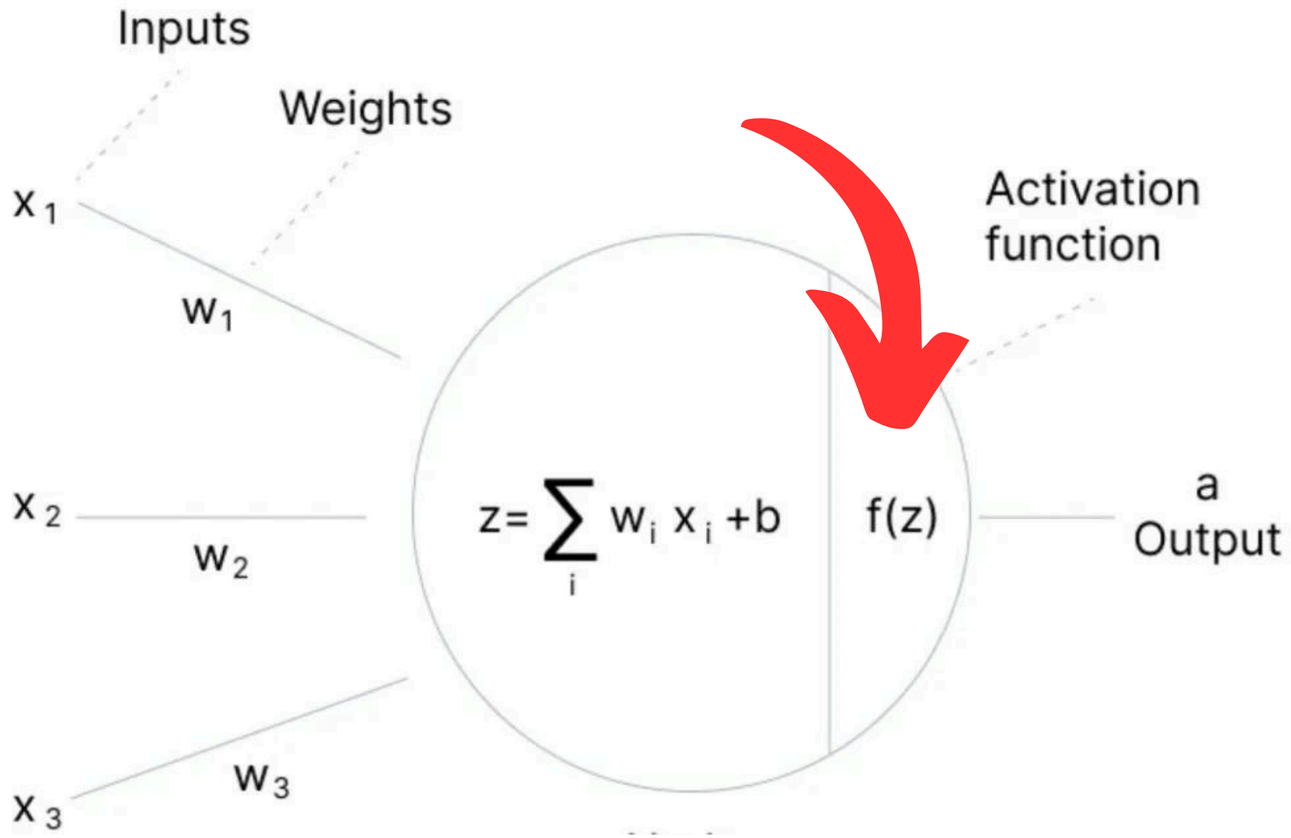
$$y_2 = w'_{21}(w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1) + w'_{22}(w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2) + w'_{23}(w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + b_3) + w'_{24}(w_{41}x_1 + w_{42}x_2 + w_{43}x_3 + b_4) + b'_2$$

결국 $y = \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_4 + \alpha$ 꼴

(아무리 쌓아도 선형)

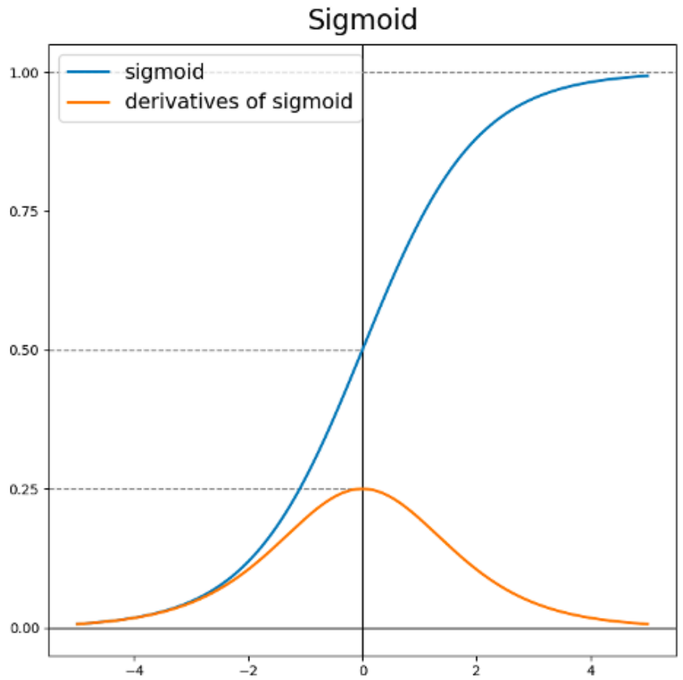
왜 단순해지죠??????

분배법칙, 교환법칙!!!!!



세포가 하는대로 해볼까..? 활성화(ACTIVATION)

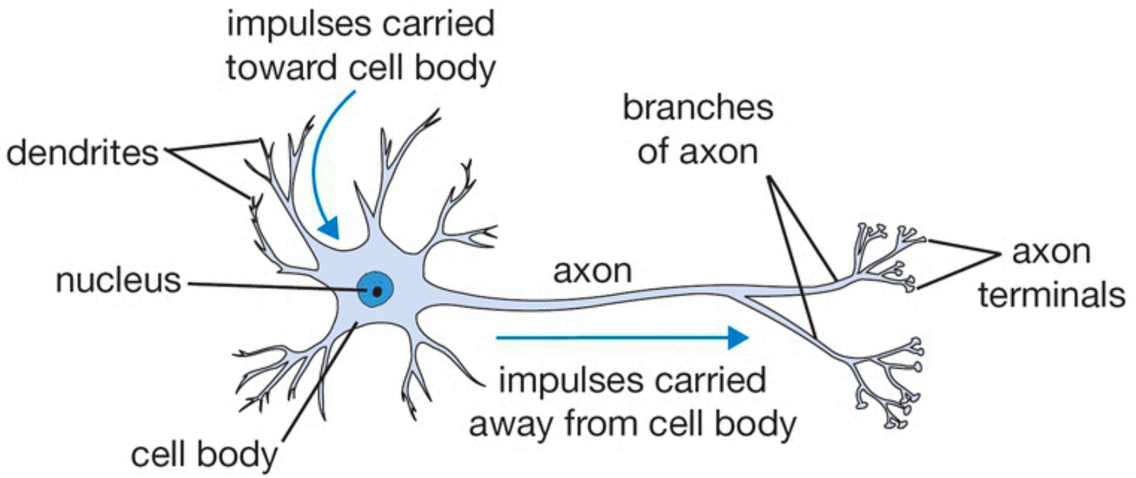
활성화(ACTIVATION): 그럼 세포는 어떻게?



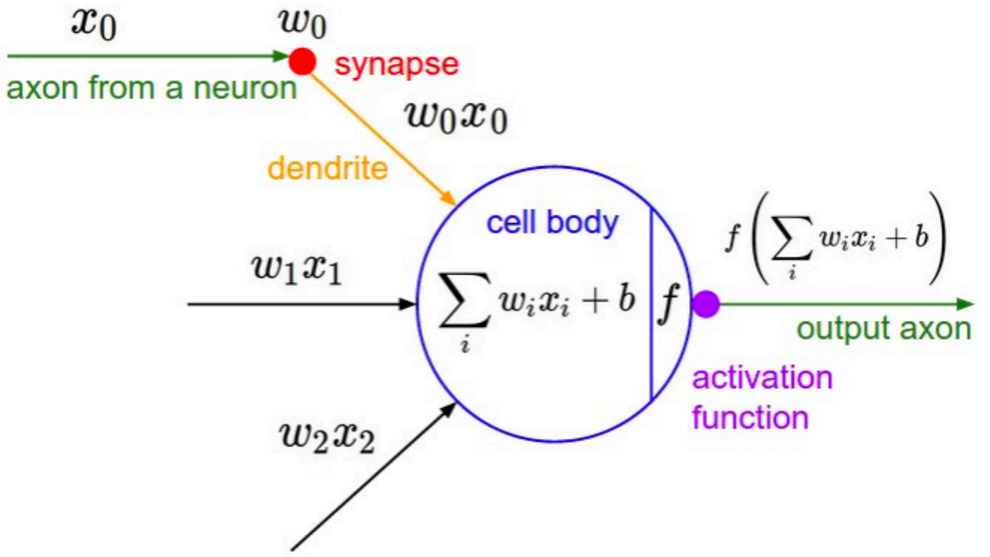
$$\sigma(z) = \frac{1}{1+e^{-z}}$$

시그모이드

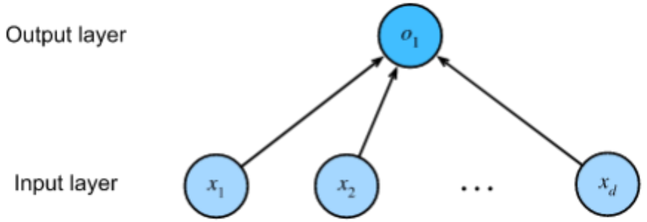
- 0~1 사이의 값으로 출력 - 이진 분류의 출력층에 사용하기 좋음 (0.5를 기준으로 분류)
- 기울기 소실 문제 - z값이 너무 작거나 큰 경우 기울기가 0으로 수렴하여 소실됨

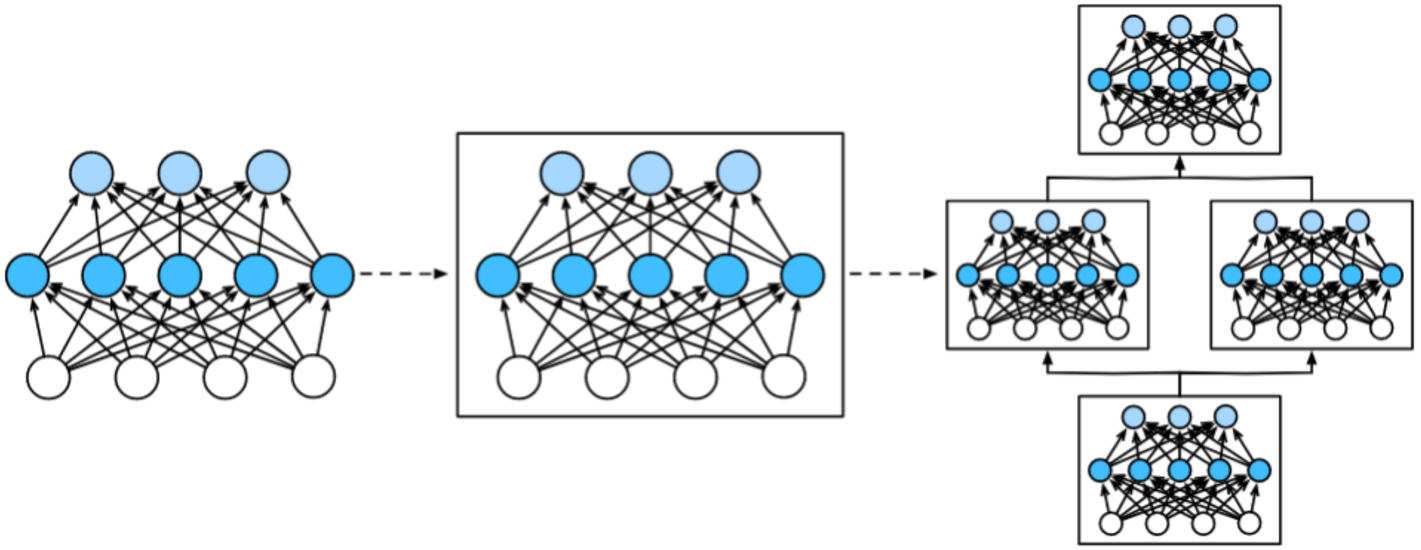


실제 뇌에서 기능하는 최소 연산 유닛인 **뉴런**



Deep learning의 최소 연산 유닛인 **perceptron**



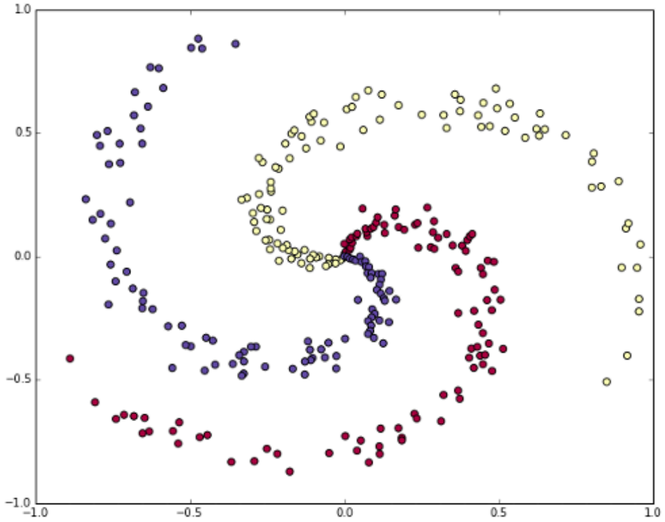


하나의 Perceptron Layer -> NN block

NN block을 여러 개 쌓아 큰 모델을 만든다

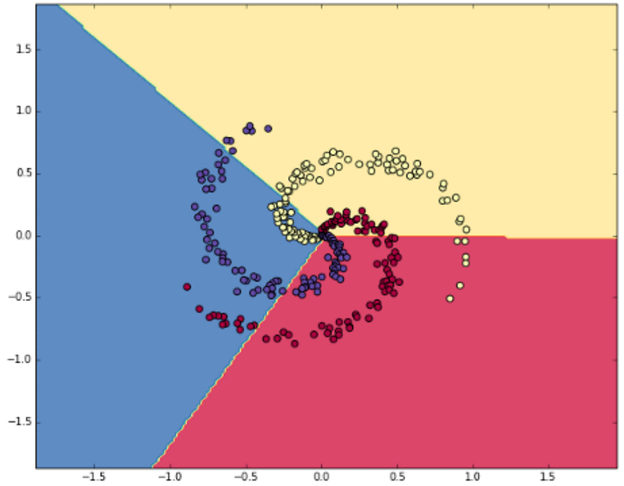
쌓을 수 있겠네!!!

선으로 가를 수 없다!

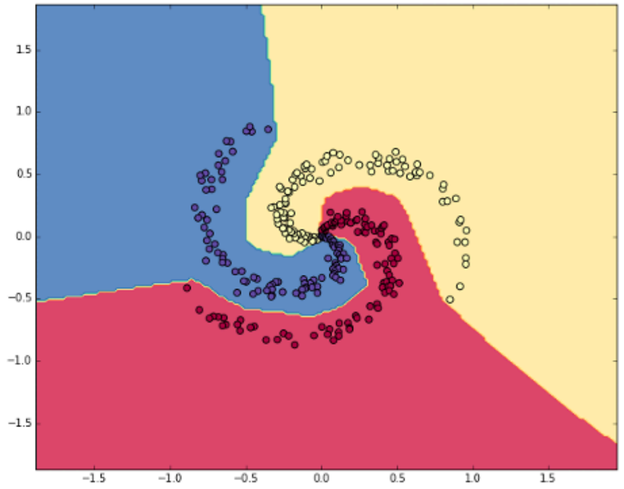


주어진 데이터

되는데요..



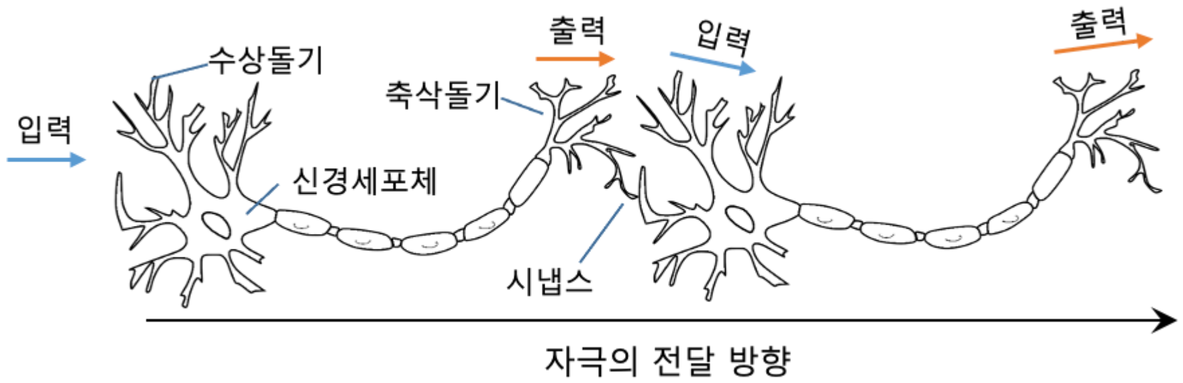
Linear decision boundary

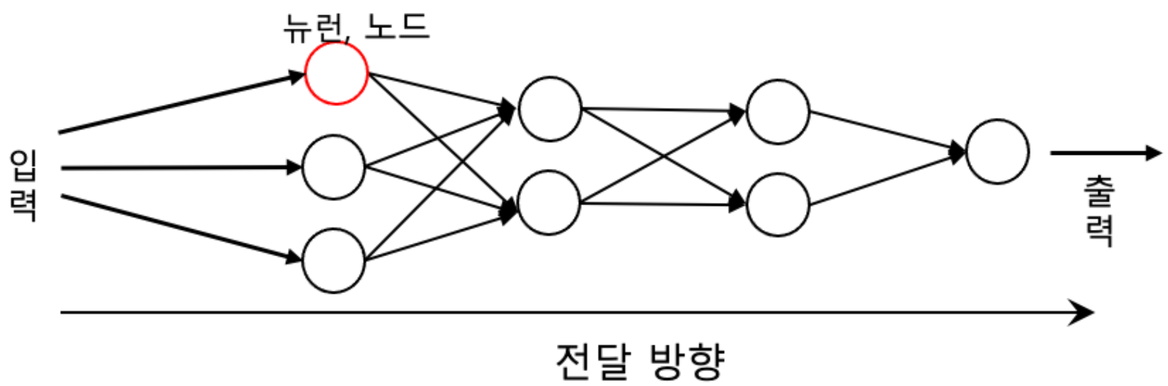


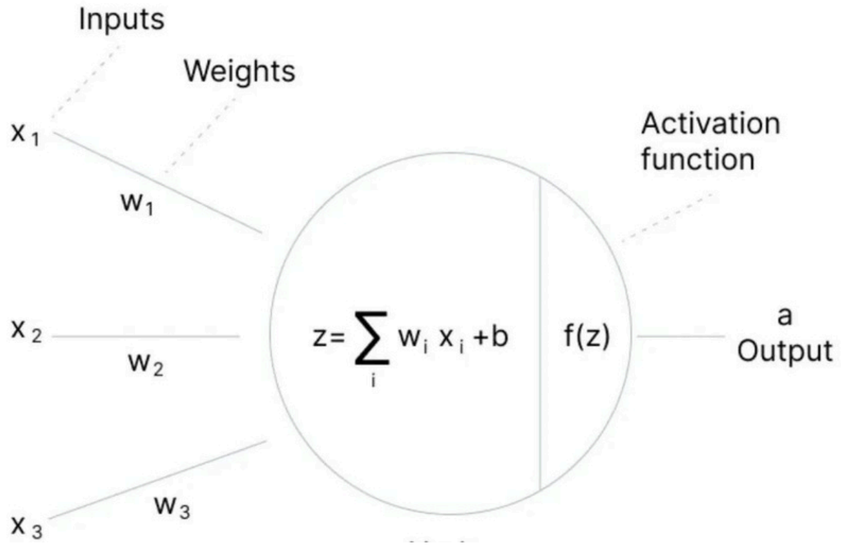
Nonlinear decision boundary

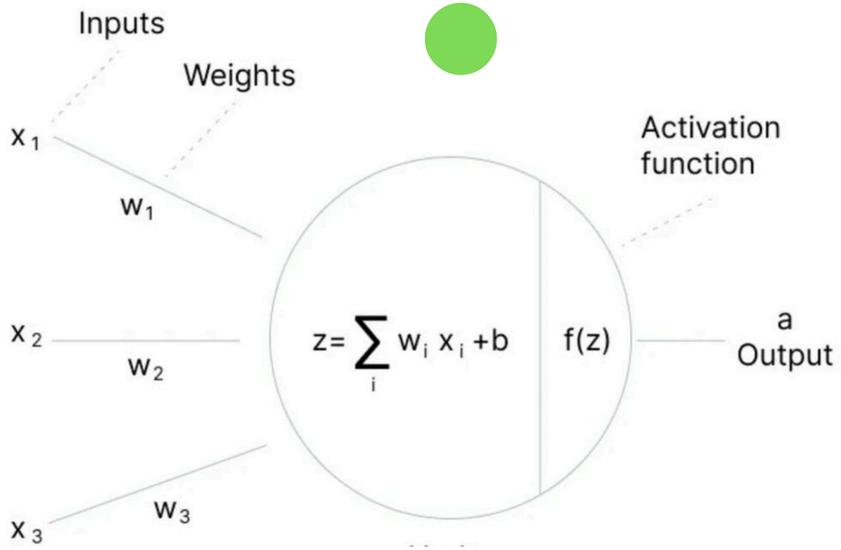
MULTI-LAYER PERCEPTRON

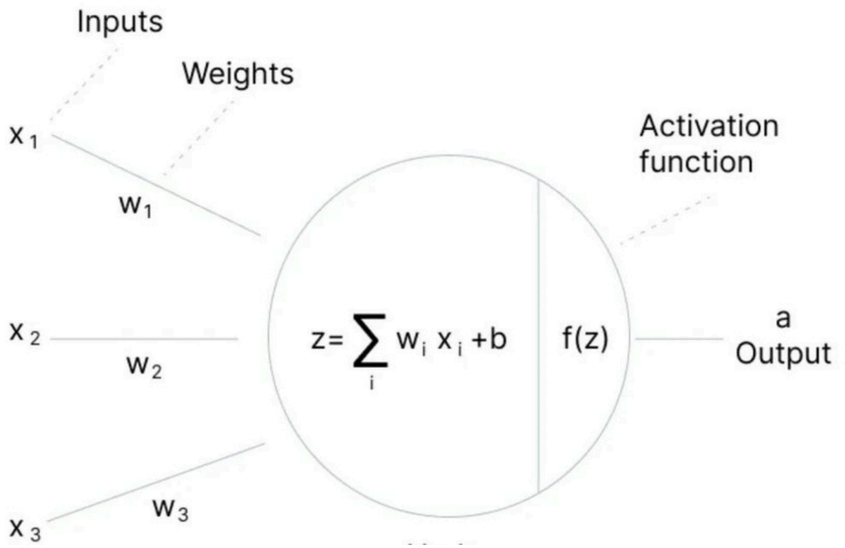
Multilayer Perceptron (MLP)





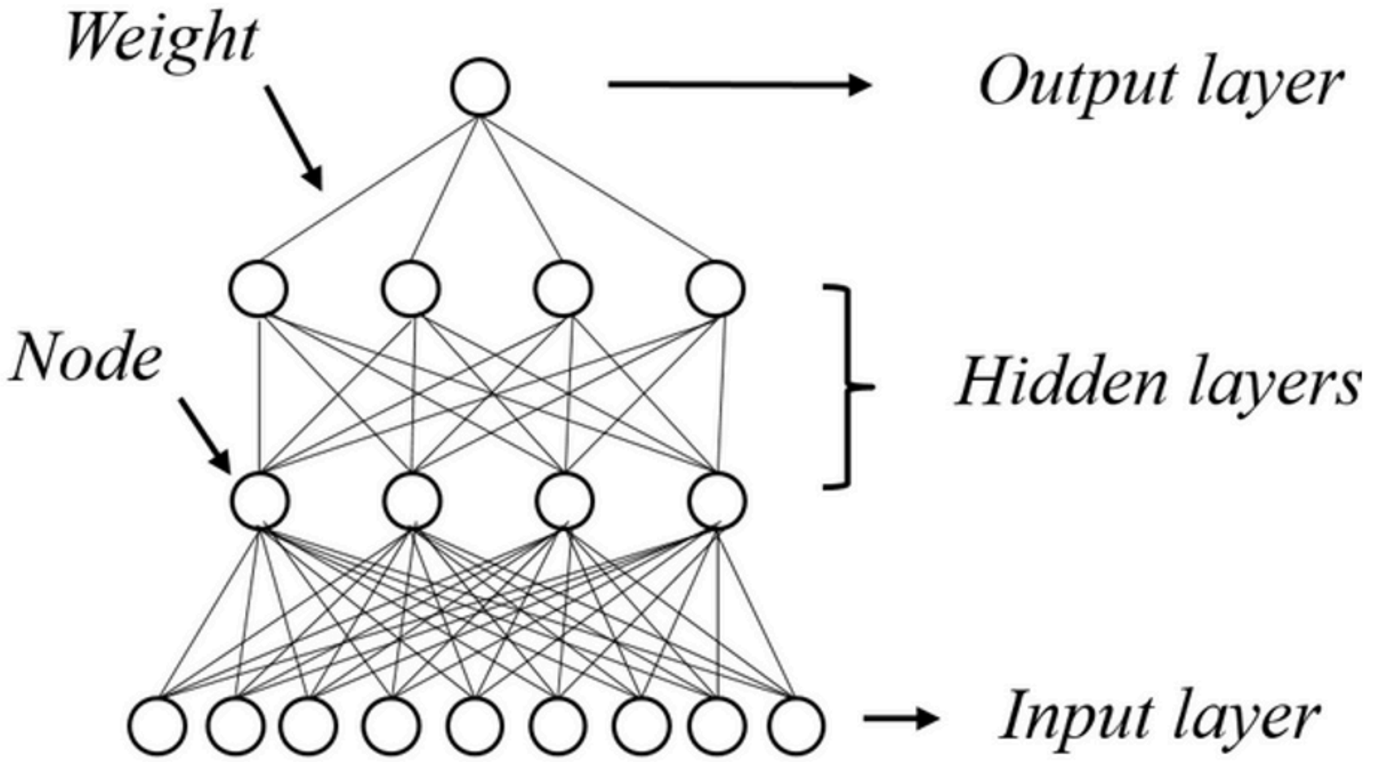






COMPUTE-R

우리가 아는 숫자
VS
우리가 알고싶은 숫자



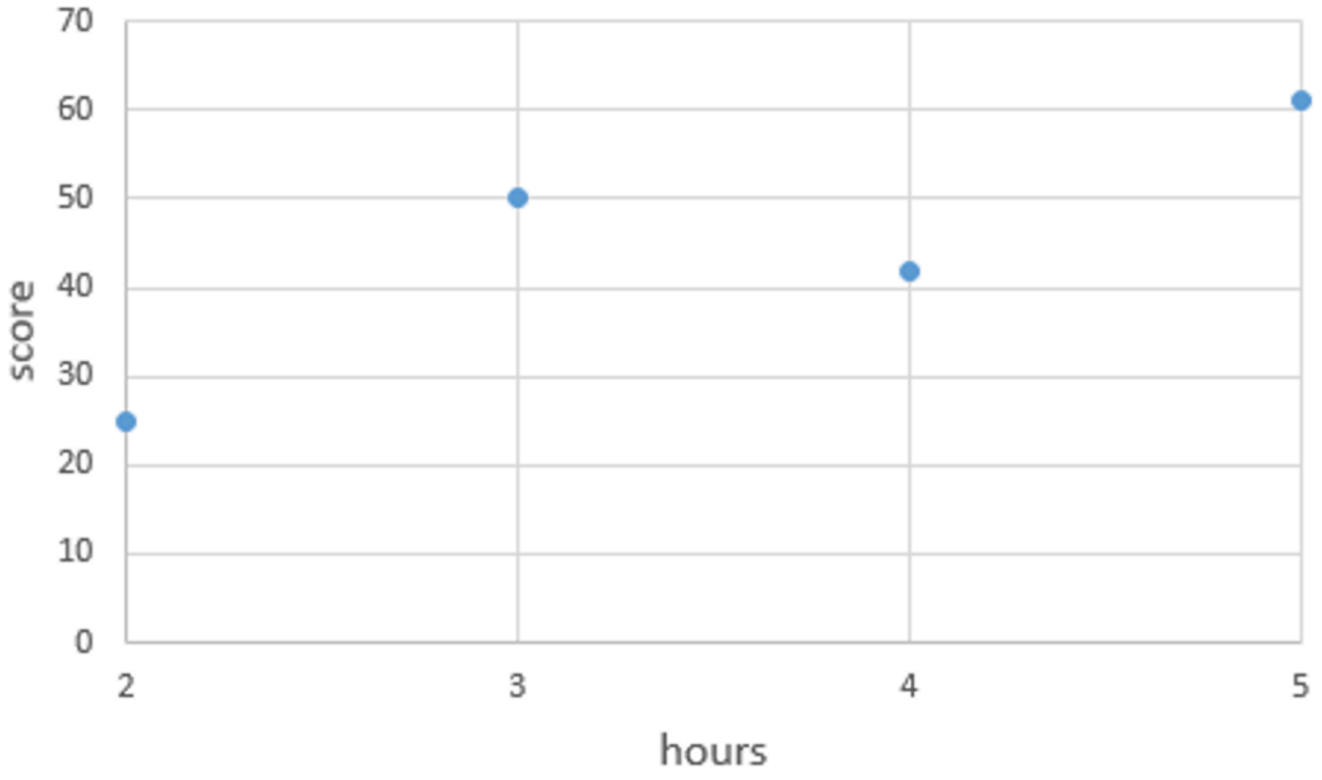
출력 데이터

여러개의 노드로 구성된 layer

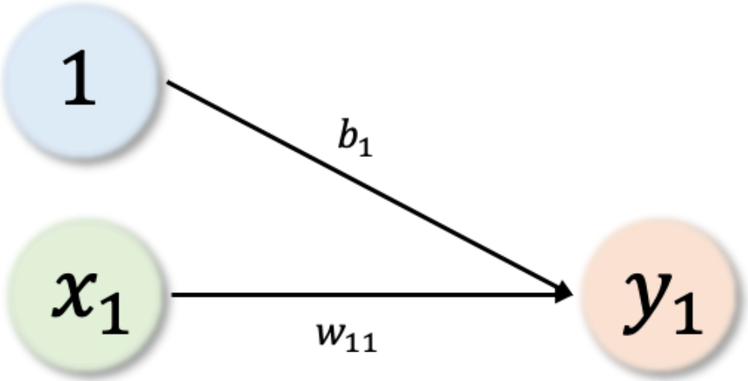
입력 데이터

목표

: 원하는 예측값을 달성하는 가중치 값(Weight)을 찾는 것



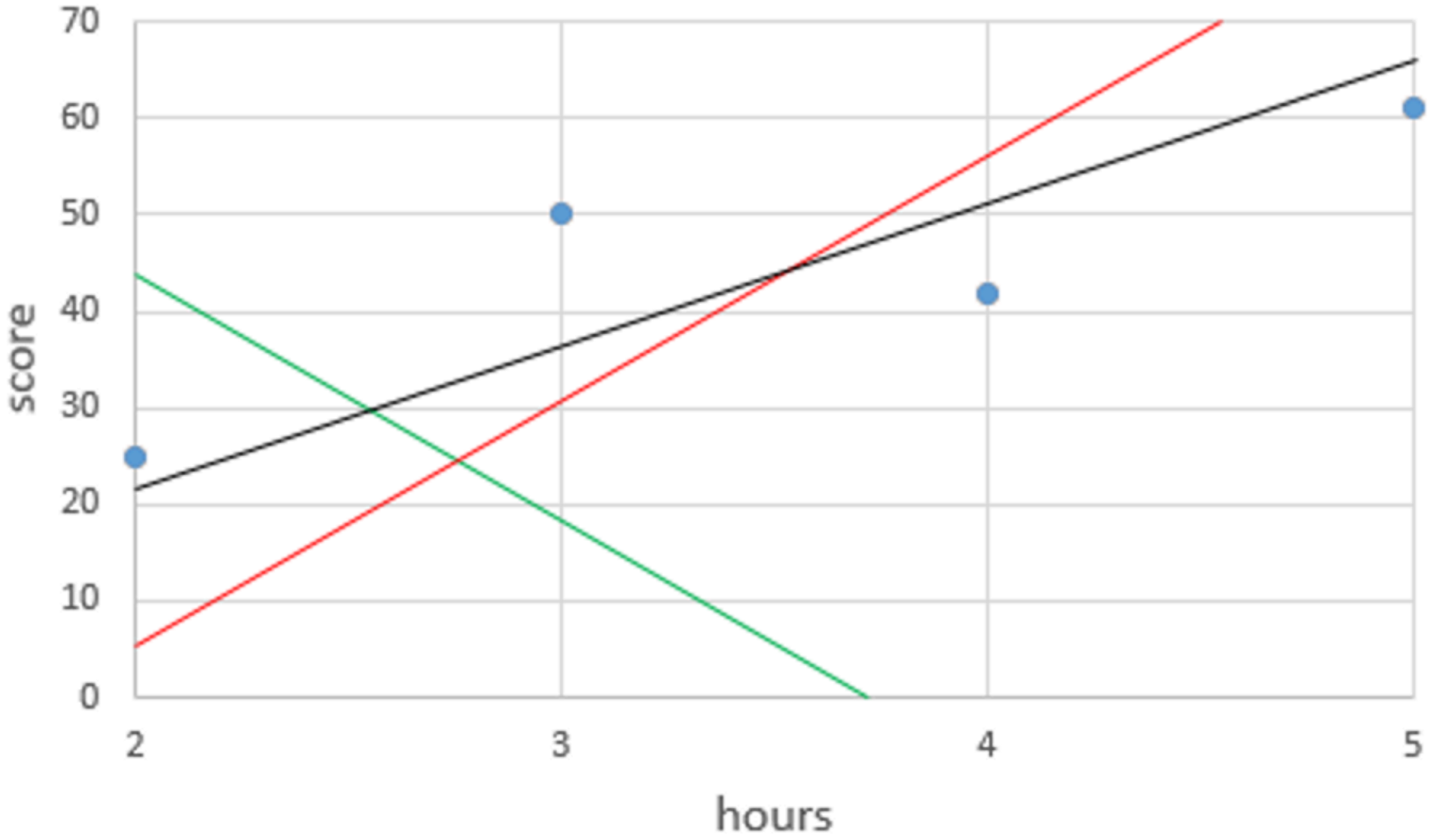
$$y_1 = w_{11}x_1 + b_1$$



1차원 1차원

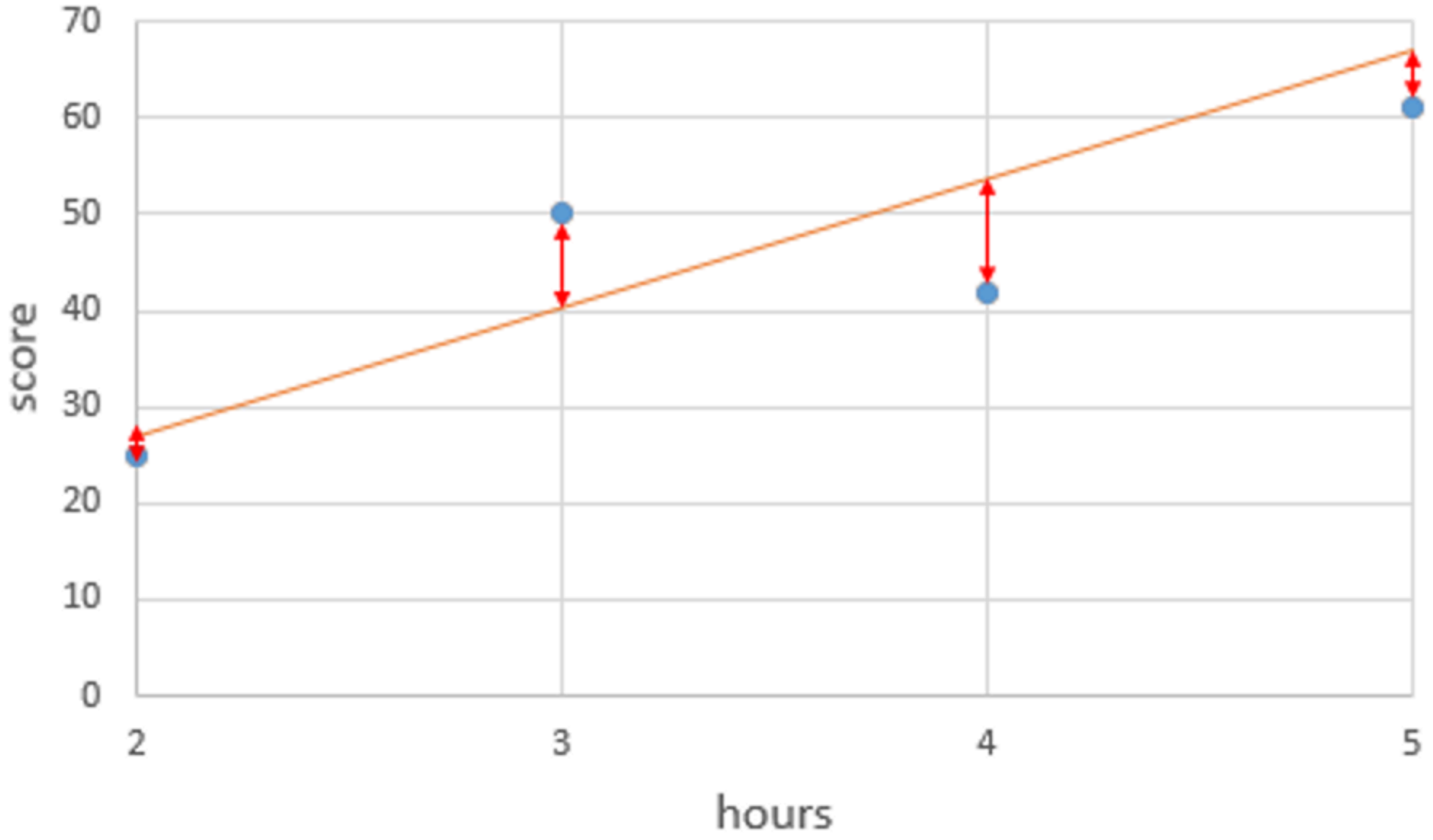
$$[w_{11}][x_1] + [b_1] = [y_1]$$

known: x=hours, y=score
find w11, b1



어떤게 좋나요?

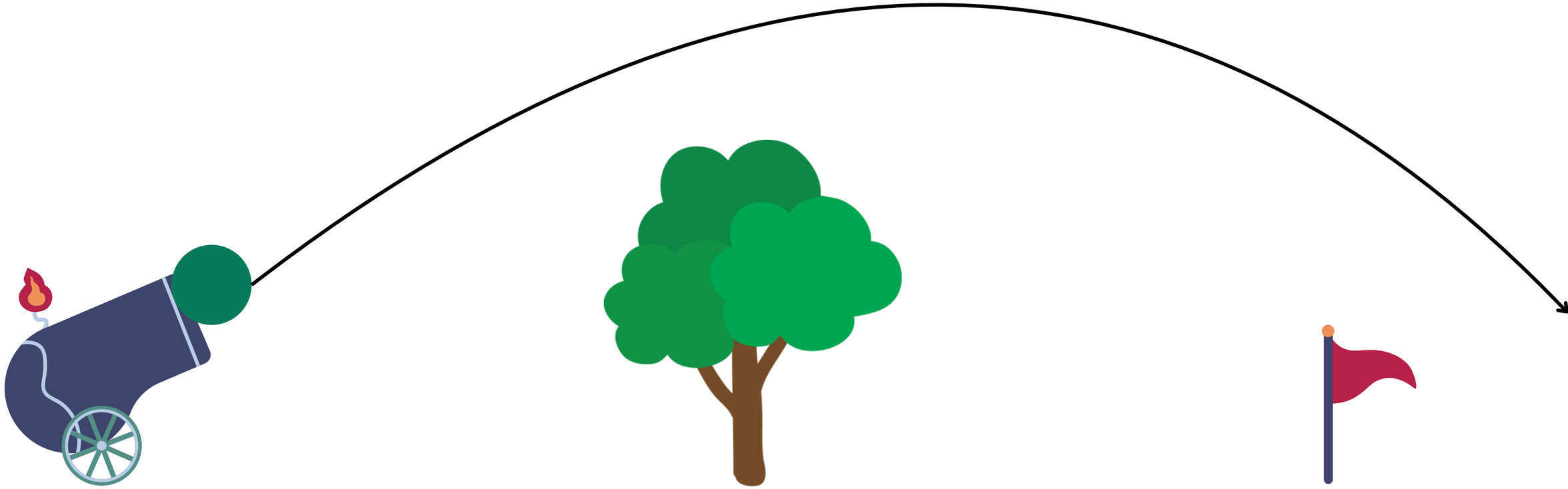
별로인가요?



아까보다 낫나요?

얼마나 더 좋아졌나요?

대포의 각도와 위치를 조절해야합니다



여러분은 어떻게 하시겠습니까?

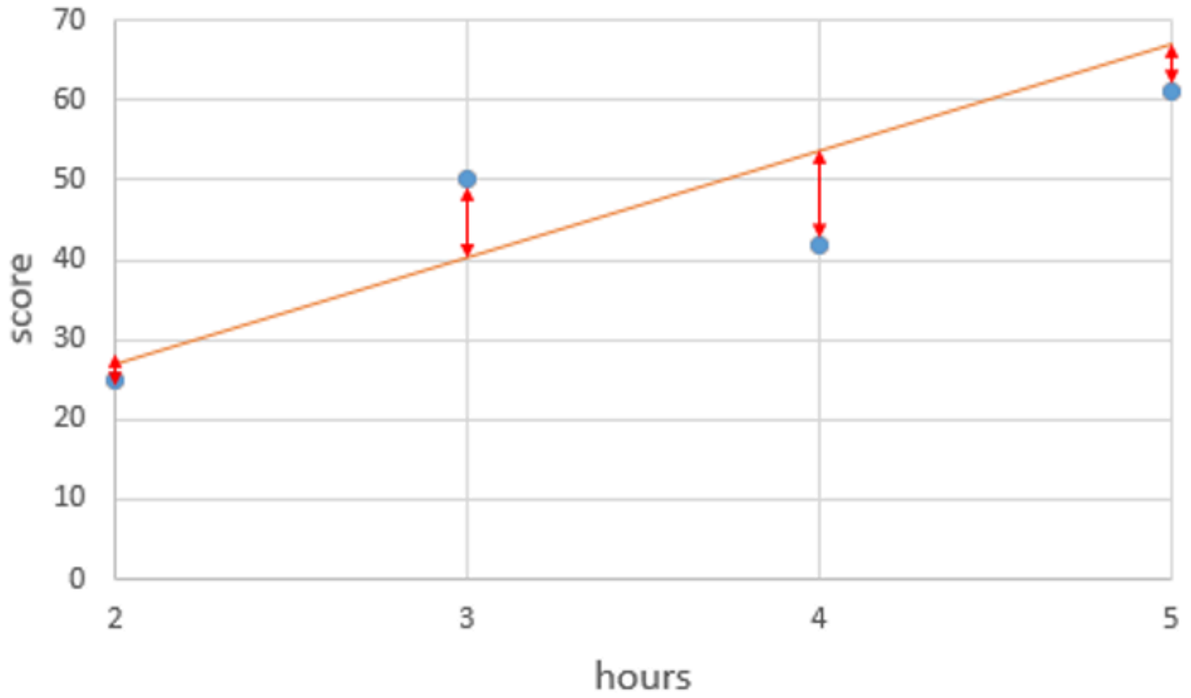
우주선 발사..

처음부터 잘 쏘면된다. 철저한 계산으로

장난감 비행기

한번 쏴보고, 오차를 수정한다

우리가 오차를 예측값과 실제값의 차이라고 정의를 한다면?



hours(x)	2	3	4	5
실제값	25	50	42	61
예측값	27	40	53	66
오차	-2	10	-9	-5

$$\sum_{i=1}^{n} \left[y^{(i)} - H(x^{(i)}) \right]^2 = (-2)^2 + 10^2 + (-9)^2 + (-5)^2 = 210$$

비용함수!!

비용 함수(cost function) = 손실 함수(loss function) = 오차 함수(error function) = 목적 함수(objective function)

$$cost(W,b) = \frac{1}{n}\sum_{i=1}^{n}\left[y^{(i)} - H(x^{(i)})\right]^2$$

“적절한 W를 찾는다”

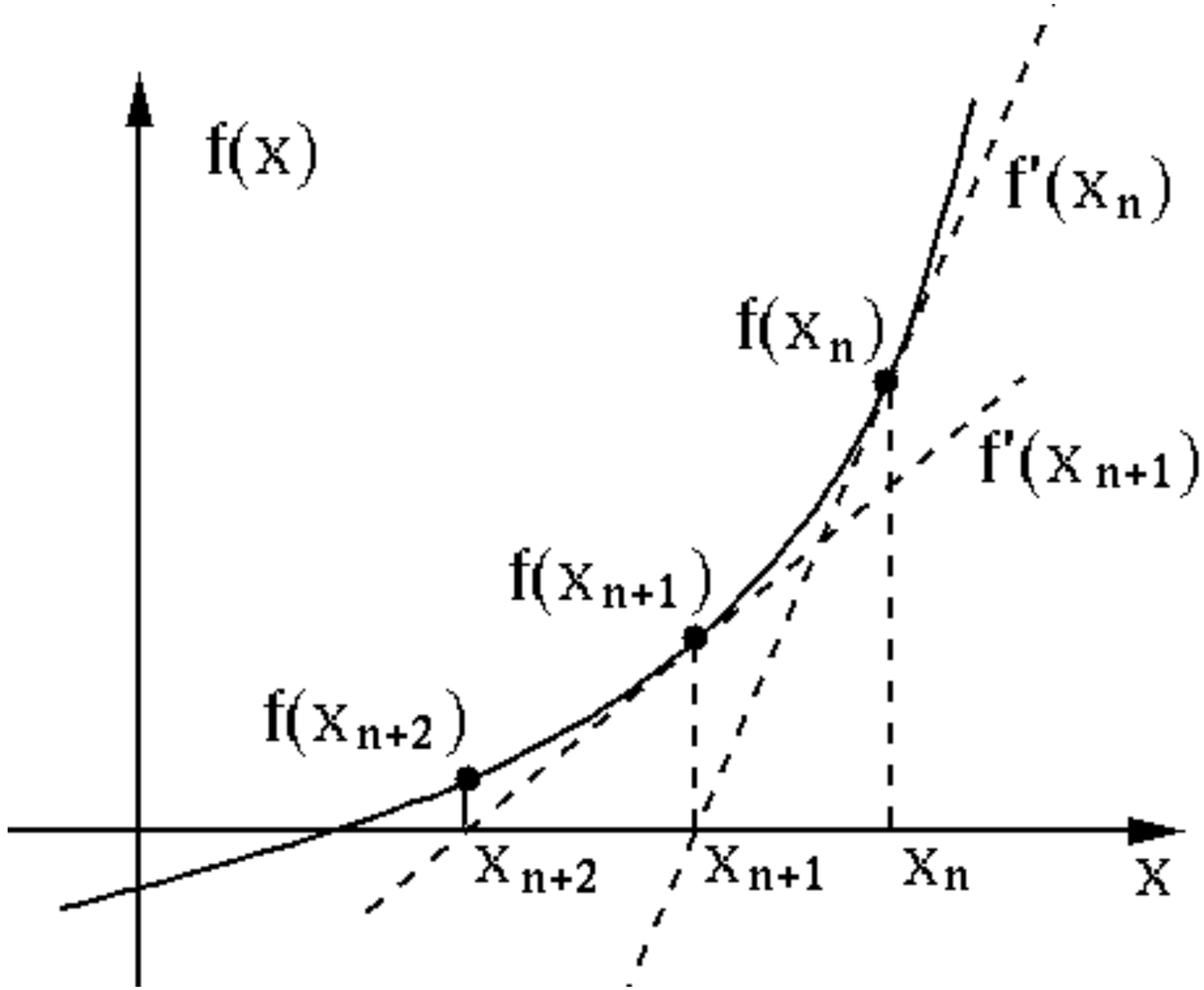


“COST가 가장 작은 W를 찾는다”

HOW?

NUMERICAL SOLUTION (수치해석학): NEWTON-RHAPSON'S METHOD

미분 = 기울기



$$y = f'(x_0)(x - x_0) + f(x_0)$$

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

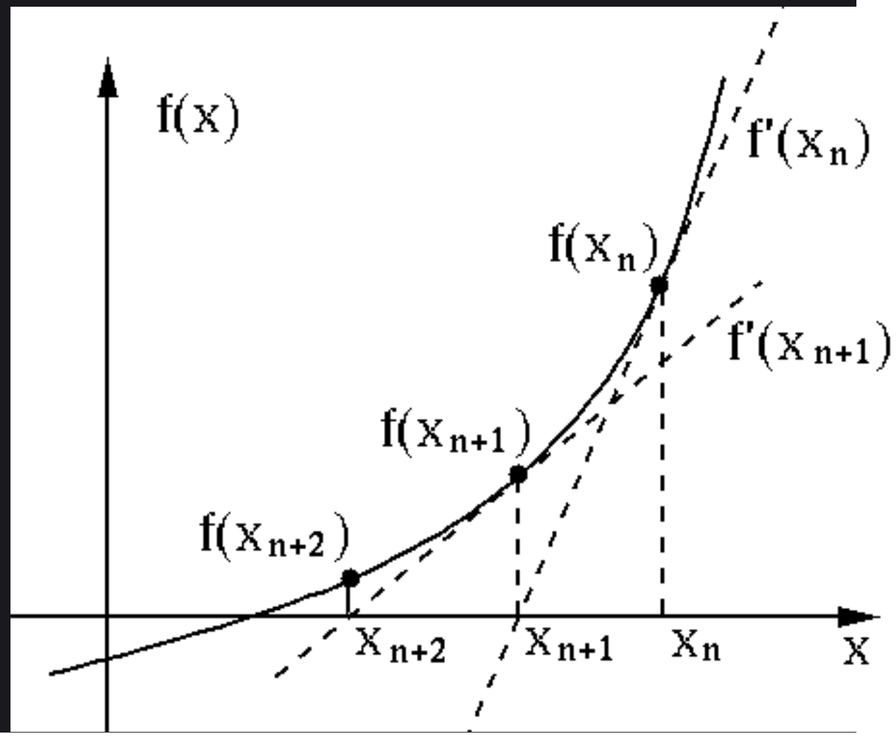
$\sqrt{2}$는 방정식 $x^2 - 2 = 0$의 한 근이다. $f\left(x\right) = x^2 - 2$로 놓으면 $f'\left(x\right) = 2x$이므로 점화식은 다음과 같다.

$$x_n = x_{n-1} - \frac{{x_{n-1}}^2 - 2}{2x_{n-1}}$$

$x_0 = 2$라고 하면 다음과 같이 계산된다. 볼드체는 실제 값과 일치하는 자릿수.

n	x_n	$\left\lvert {x_n}^2 - 2 \right\rvert$
0	2	2
1	**1.5**	0.25
2	**1.41**666666666667	0.006944444444445
3	**1.41421**568627451	$6.00730488287127 \times 10^{-6}$
4	**1.41421356237**469	$4.51061410444709 \times 10^{-12}$

근에 빠른 속도로 수렴하는 것을 볼 수 있다.



HTTPS://NAMU.WIKI/W/%EB%89%B4%ED%84%B4-%EB%9E%A9%EC%8A%A8%20%EB%B0%A9%EB%B2%95

$\sqrt{2}$는 방정식 $x^2 - 2 = 0$의 한 근이다.

양변에 +2

거듭제곱이니까 루트를 양변에 씌우면!

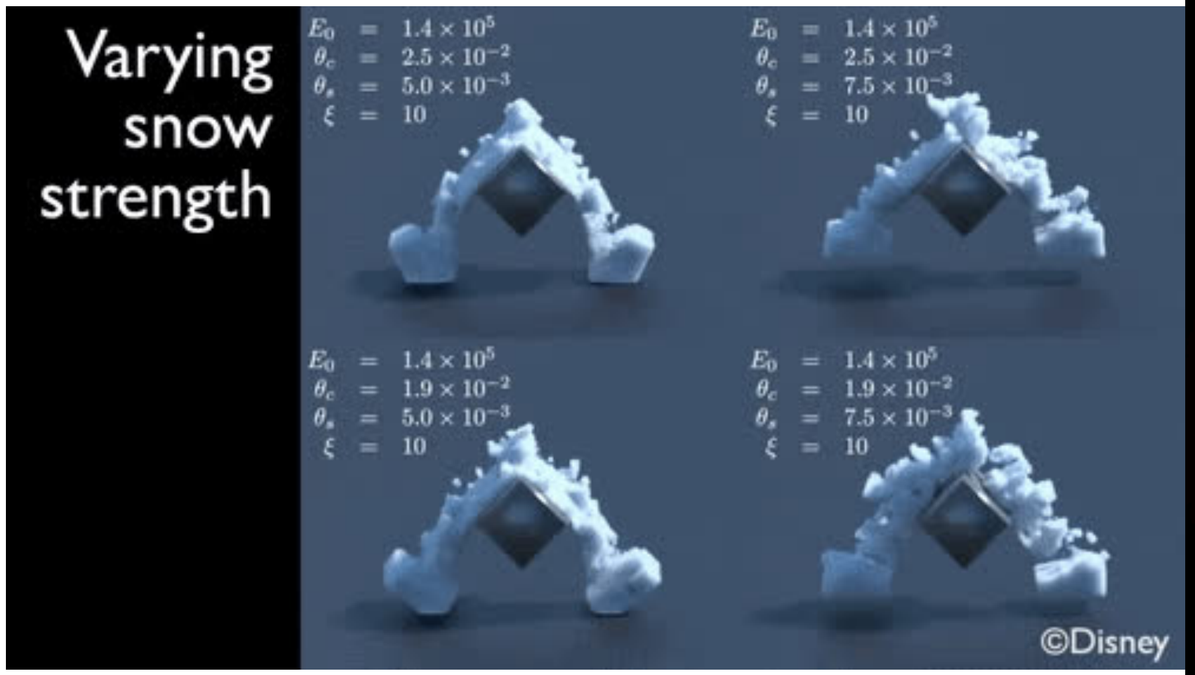
플러스 마이너스 루트 2

이런 불편한걸 왜 씀?

심지어 근사값(APPROXIMATION).. 의미 없는거 아닌가요?



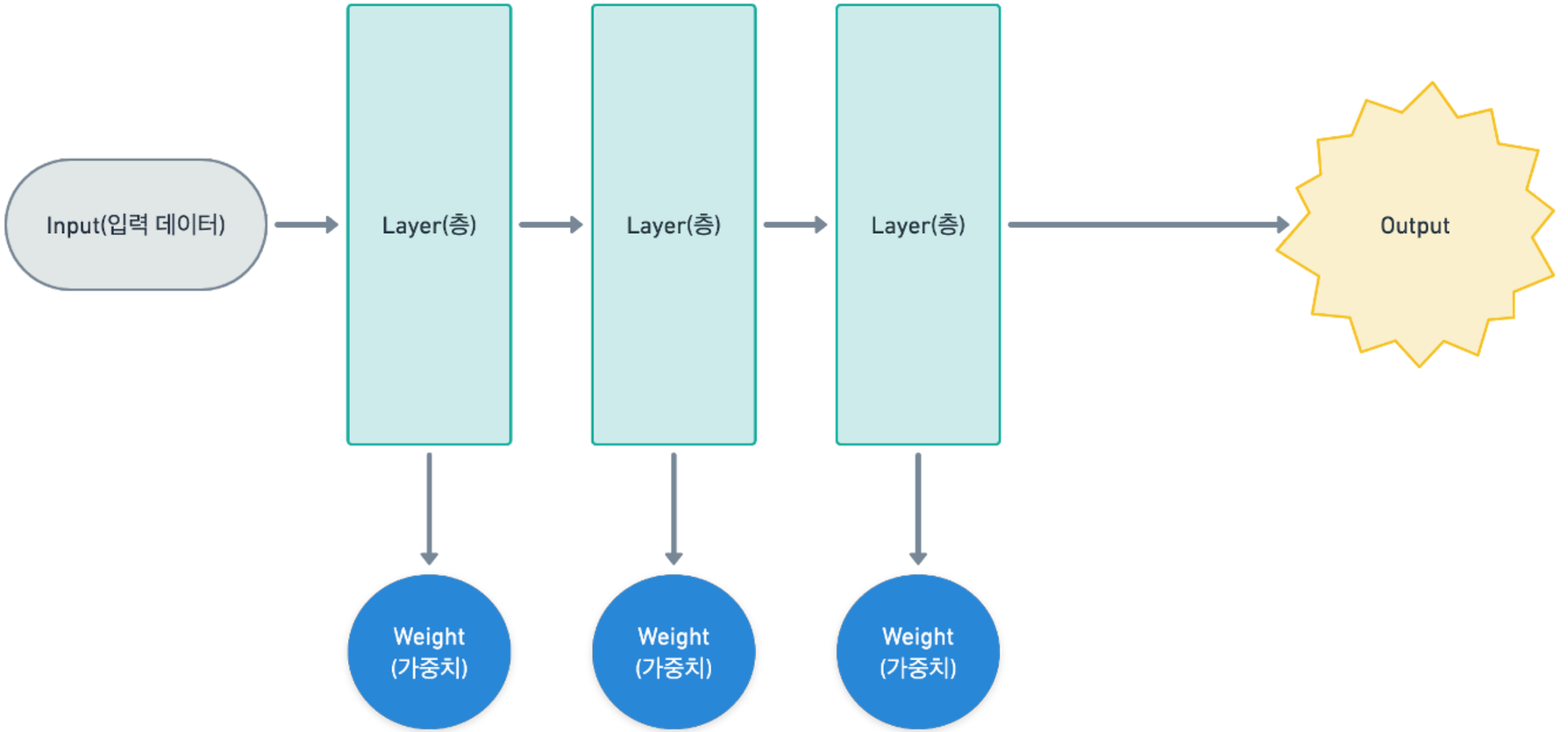
지금까지 풀이가능한 것만 배운것





APPROXIMATION이 대부분..

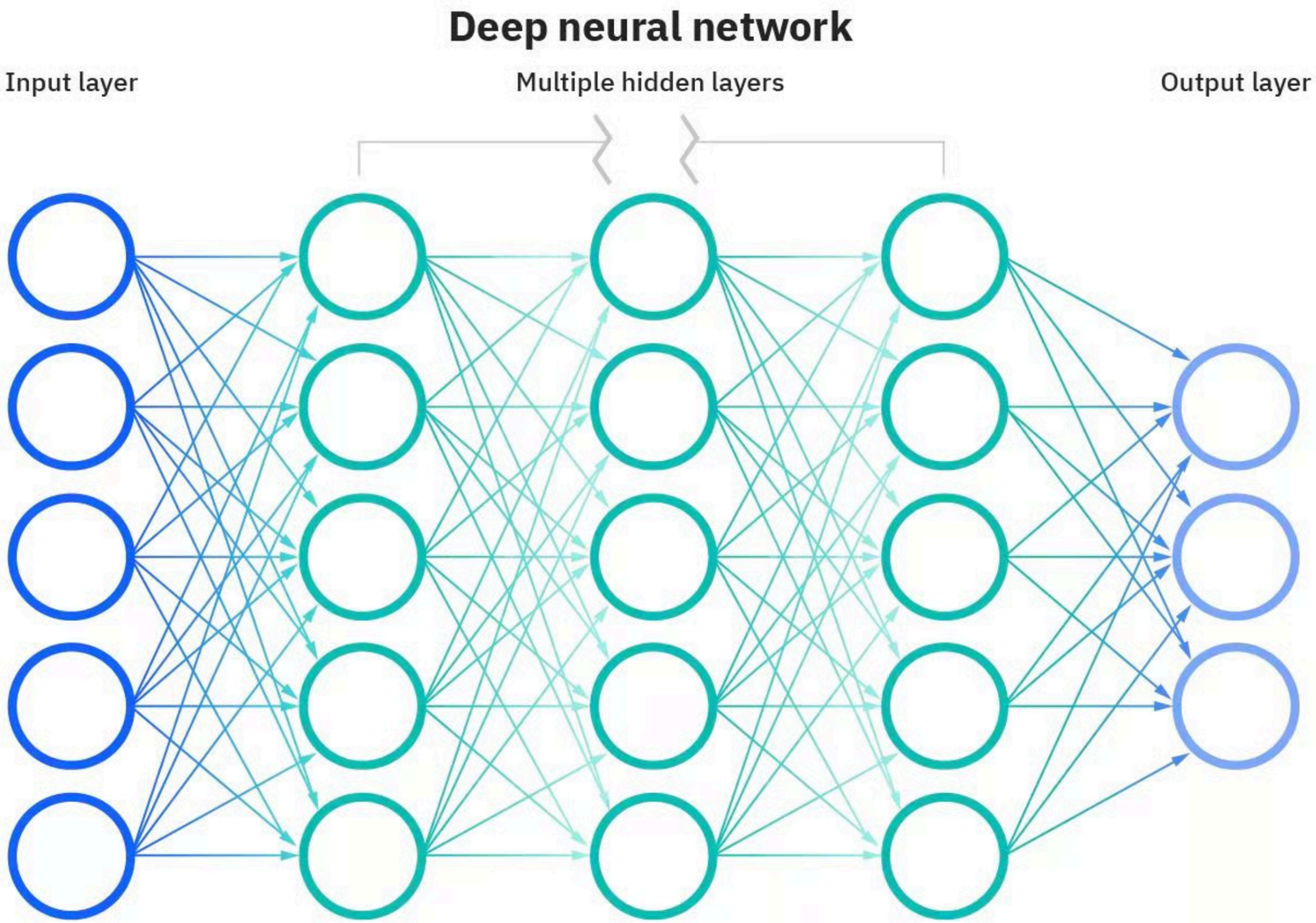

과학쿠키
풀기만 해도 12억 | 밀레니엄 7대 난제! 나비에-스톡스 방정...
과학쿠키 Science Cookies Share
그냥 풀기만 해도
12억 주는 문제?
Watch on YouTube



목표

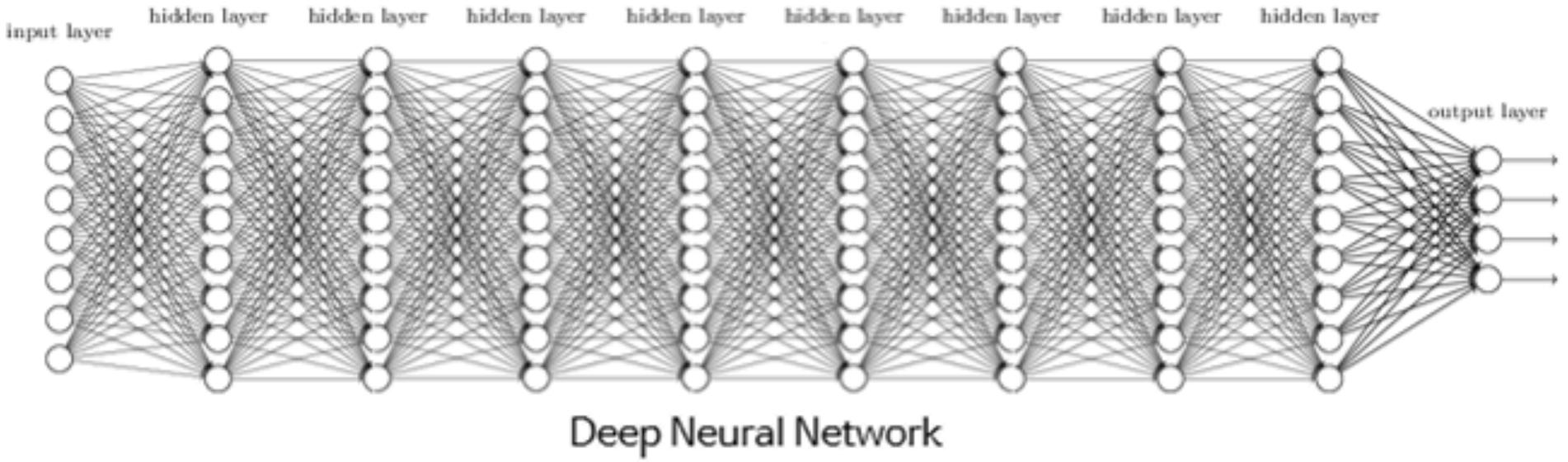
: 원하는 예측값을 달성하는 가중치 값을 찾는 것

대충

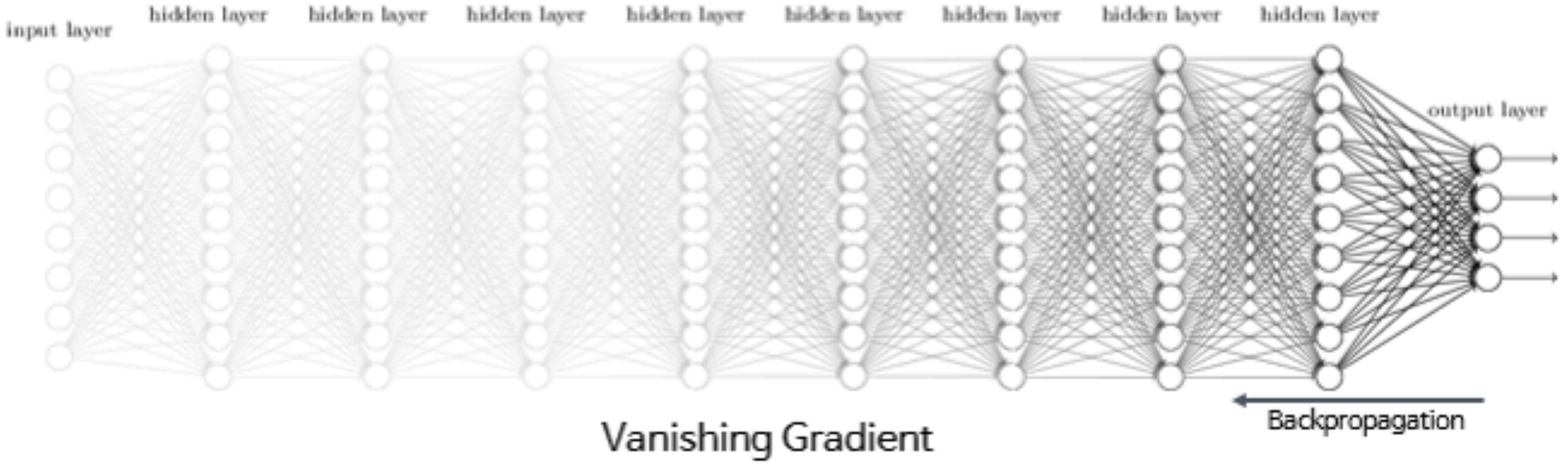

Deep neural network
Input layer
Multiple hidden layers
Output layer

히힛... 더 깊게...

GRDIENT VANISHING

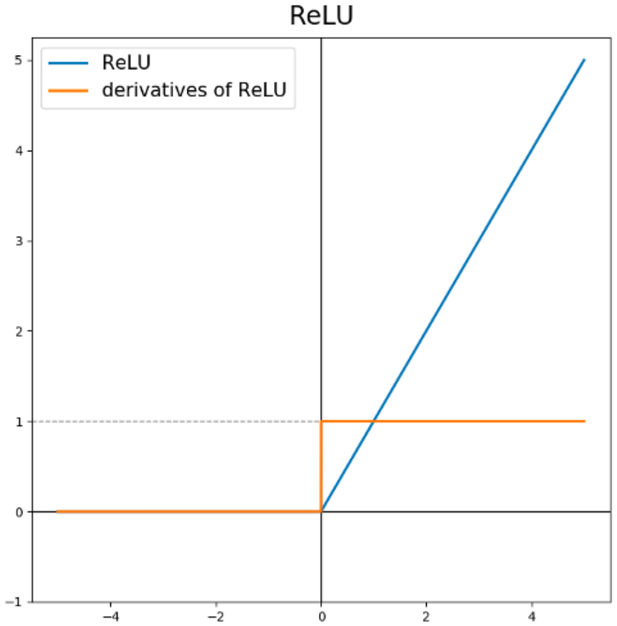


Deep Neural Network



Vanishing Gradient

RECTIFIED LINEAR UNITS IMPROVE RESTRICTED BOLTZMANN MACHINES



$$ReLU(Z) = \max(0, Z)$$

렐루

Z가 양수이면 Z를, 음수이면 0을 출력
Z가 양수인 경우 기울기가 다른 활성화 함수에 비해 큼
- 신경망의 학습 속도가 빨라짐

여러가지 ACTIVATION FUNCTION

Sigmoid



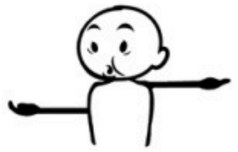
$$y = \frac{1}{1+e^{-x}}$$

Tanh



$$y = \tanh(x)$$

Step Function



$$y = \begin{cases} 0, & x<n \\ 1, & x \geq n \end{cases}$$

Softplus



$$y = \ln(1+e^{x})$$

ReLU



$$y = \begin{cases} 0, & x<0 \\ x, & x \geq 0 \end{cases}$$

Softsign



$$y = \frac{x}{(1+|x|)}$$

ELU



$$y = \begin{cases} \alpha(e^{x}-1), & x<0 \\ x, & x \geq 0 \end{cases}$$

Log of Sigmoid



$$y = \ln\left(\frac{1}{1+e^{-x}}\right)$$

Swish



$$y = \frac{x}{1+e^{-x}}$$

Sinc



$$y = \frac{\sin(x)}{x}$$

Leaky ReLU



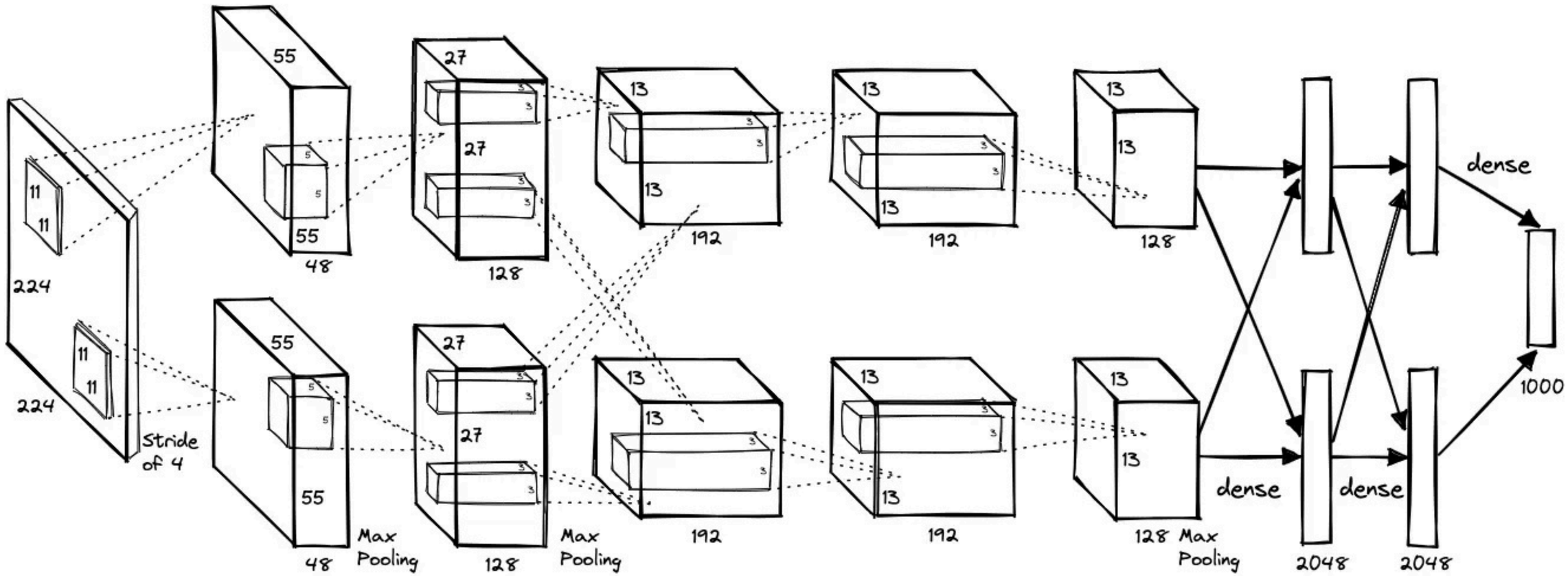
$$y = \max(0.1x, x)$$

Mish



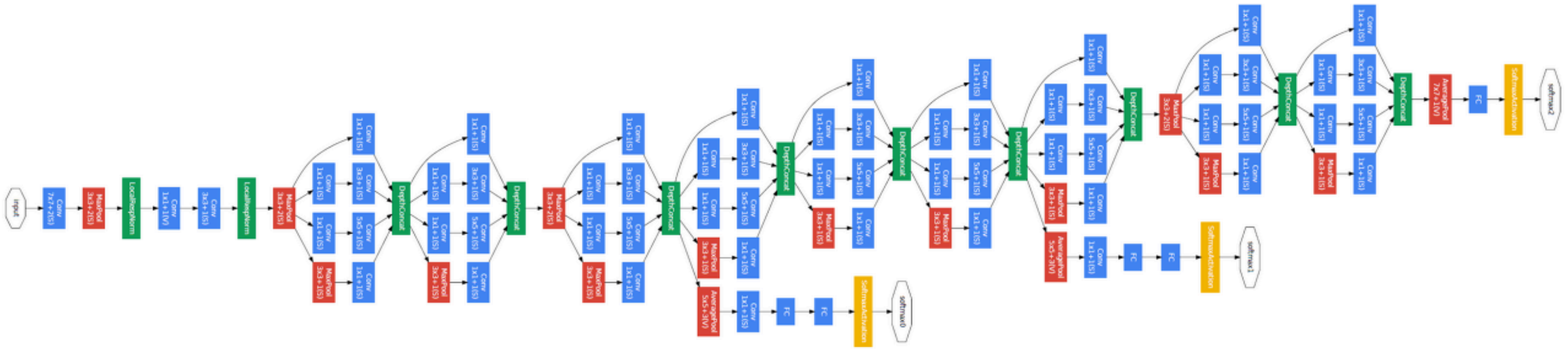
$$y = x(\tanh(\text{softplus}(x)))$$

ALEXNET

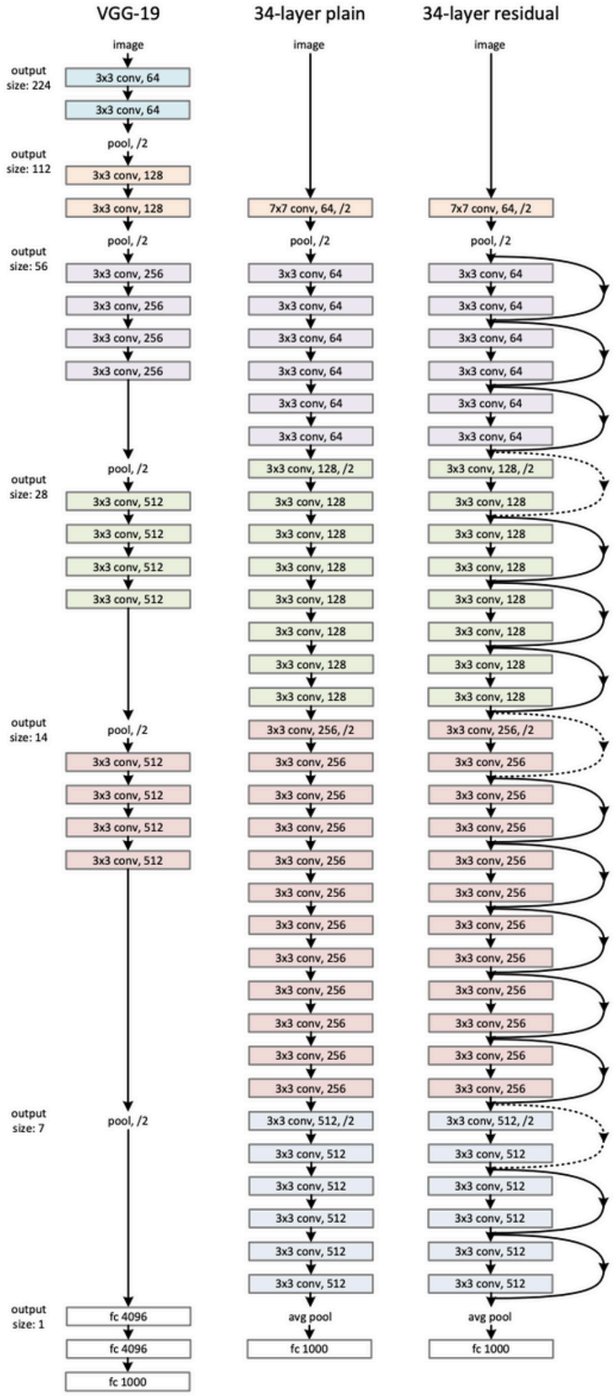


무려 두개의 GPU를 사용.
RELU가 있으니까 너무 학습이 잘되요!!

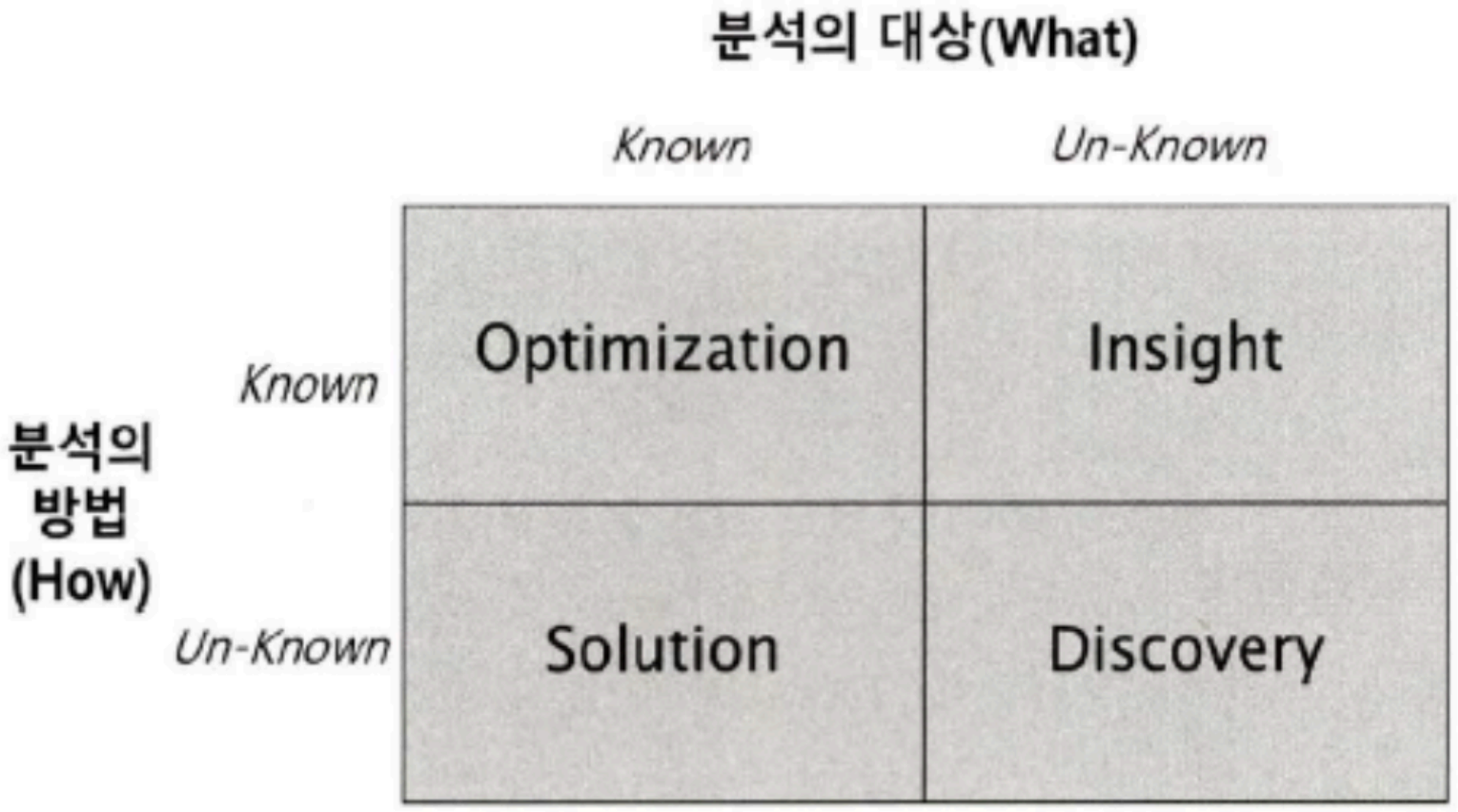
GOOGLENET: GOING DEEPER WITH CONVOLUTIONS



더 깊게!
더 크게!



Y=WX 인데 학습을 하니까
W가 1로는 학습이 잘 안되네..
1을 학습하는 대신 지름길을 뚫어주자...


분석의 대상(What)
Known
Un-Known
분석의 방법 (How)
Known
Un-Known
Optimization
Insight
Solution
Discovery